Deploy a .NET 8 App to Azure Kubernetes Service (AKS) – Tutorial Guide

Kosisochukwu Ugochukwu

A beginner-friendly guide to deploying a .NET app to the cloud using Docker and Kubernetes.

Deploying applications to the cloud can seem daunting, especially if you are new to containers and Kubernetes. This guide is designed for beginners and walks you step by step through building a simple .NET 8 Web API, packaging it in a Docker container, and deploying it to Azure Kubernetes Service (AKS). By the end, you will also have automated deployments via GitHub Actions, so any code push instantly updates your app in the cloud.

Think of it as a hands-on way to learn modern cloud deployment, without getting lost in complex setups. By the time you finish, you will not only understand the tools but also gain confidence in deploying real-world apps to the cloud.

What You will Learn

By the end of this tutorial, you will know how to:

Build a .NET 8 Web API

Containerize your app with Docker

Deploy containers to Kubernetes in the cloud

Set up automated deployments with GitHub Actions

What You will be able Build :

Simple Weather API – Returns fake weather data (just like a real weather app)

Containerized App – Runs in Docker containers

Cloud Deployment – Hosted on Azure Kubernetes Service (AKS)

Automatic Updates – Push code to GitHub → auto-deploys to Azure

Required Tools to Set Up Your Computer

Install the following:

Visual Studio Code – code.visualstudio.com

.NET 9 SDK – dotnet.microsoft.com

Docker Desktop – docker.com/products/docker-desktop

Start Docker Desktop after installing

Azure CLI – docs.microsoft.com/cli/azure/install-azure-cli

kubectl (Kubernetes CLI) – kubernetes.io/docs/tasks/tools/

GitHub CLI – cli.github.com

Git – git-scm.com

Required Accounts

GitHub Account – github.com

Azure Account – azure.microsoft.com/free

Test Your Setup

Run these commands in your terminal (Command Prompt on Windows, Terminal on Mac/Linux) to verify:

```
dotnet --version
docker --version
az --version
kubectl version --client
gh --version
git --version
```

You should see version numbers for all tools. If anything fails, reinstall that tool.

Step 1: Create the .NET Application

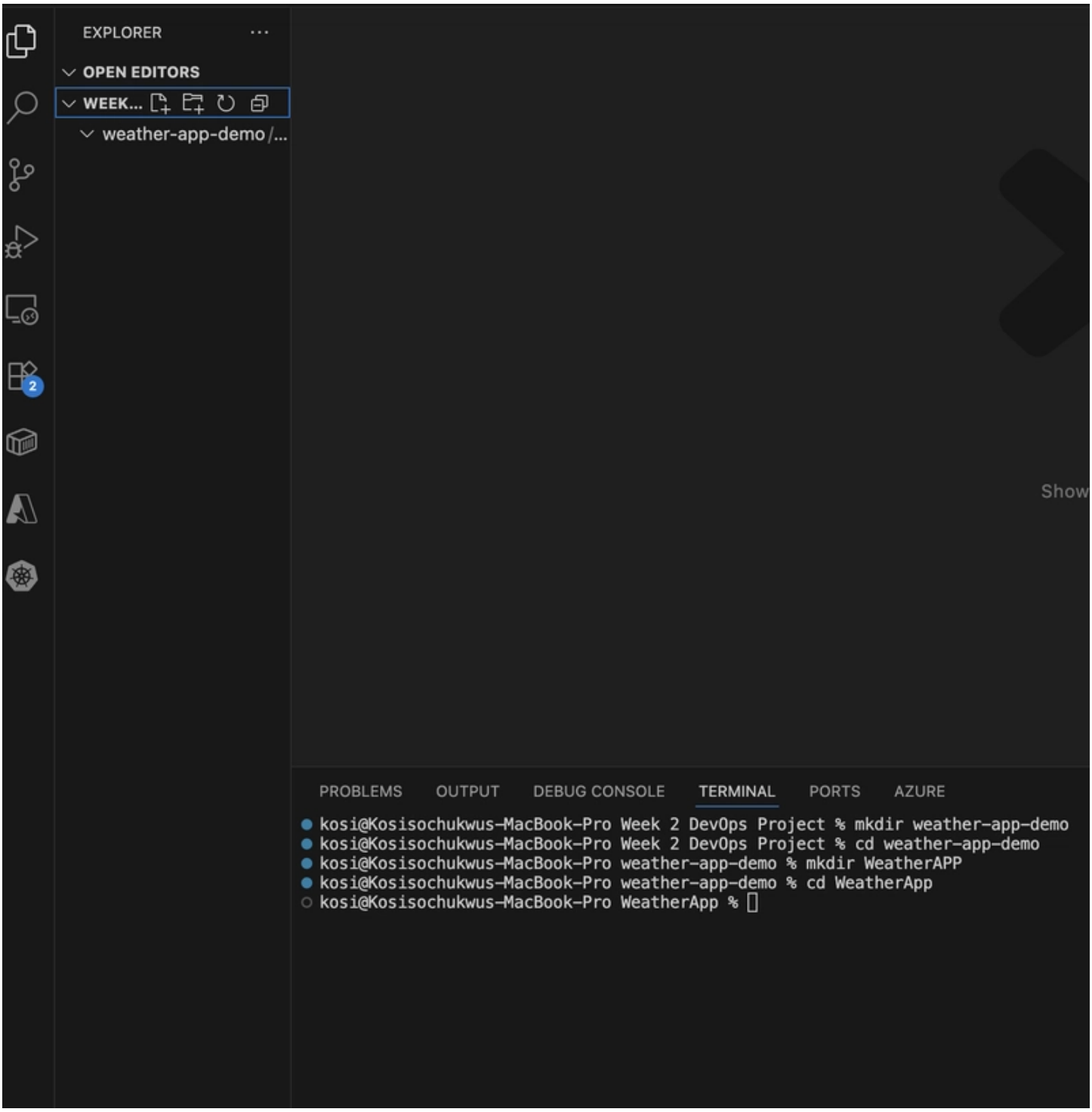
Let's start by building a simple .NET 8 Web API that we will later containerize and deploy to Azure.

1.1 Set Up Project Structure

Create a new folder for your app:

```
mkdir weather-app-demo
```

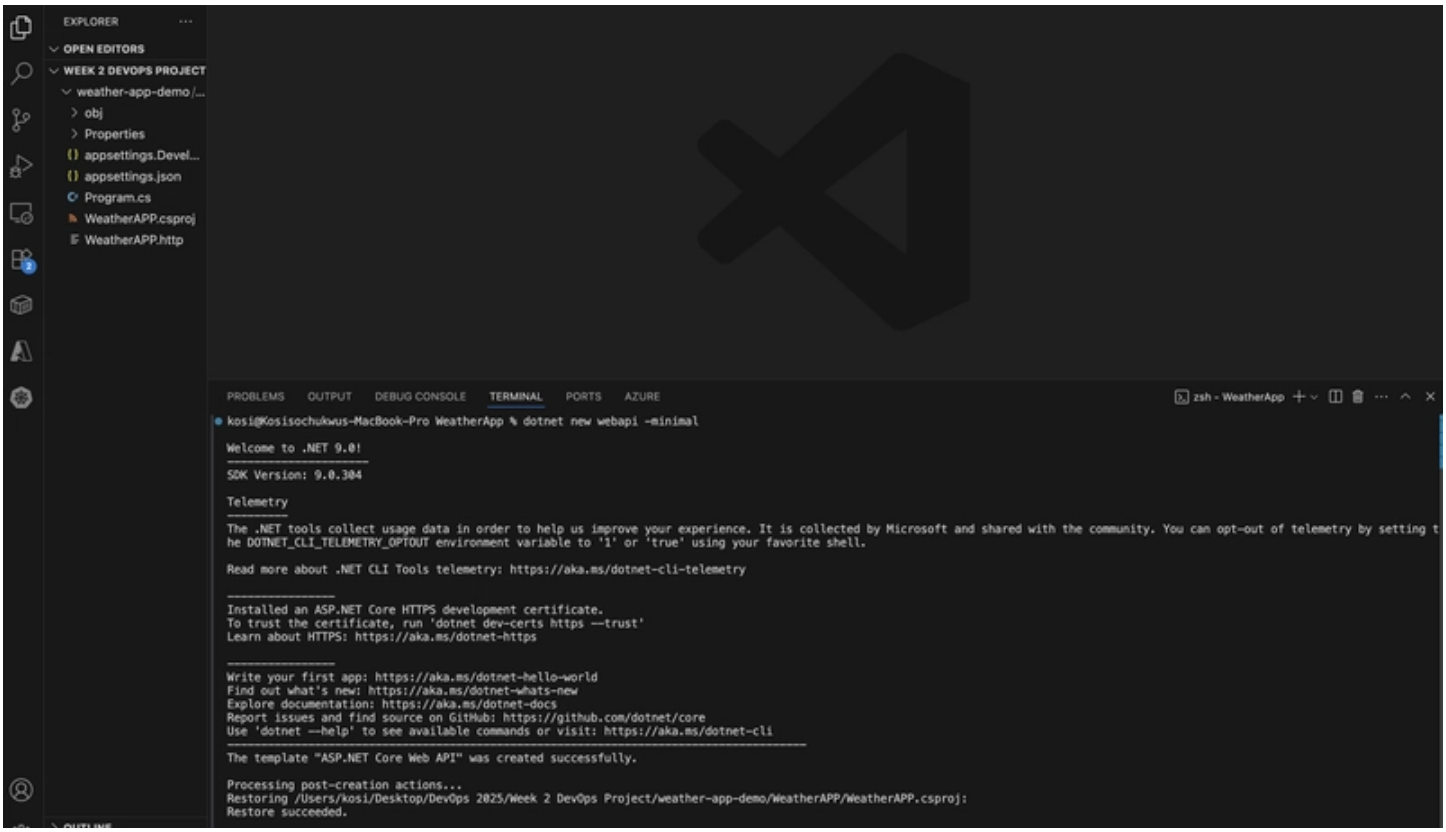
```
cd weather-app-demo
mkdir WeatherApp
cd WeatherApp
```



1.2 Initialize a .NET Project

Generate a minimal Web API template:

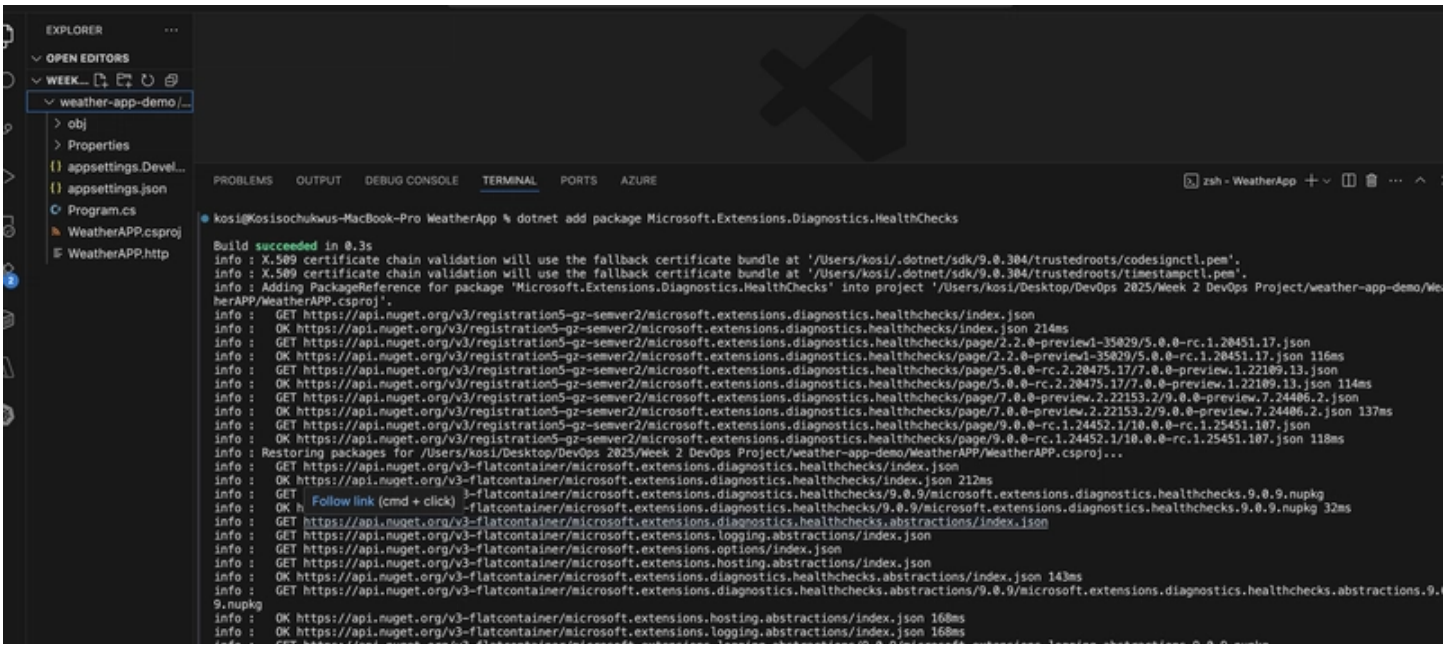
```
dotnet new webapi -minimal
```

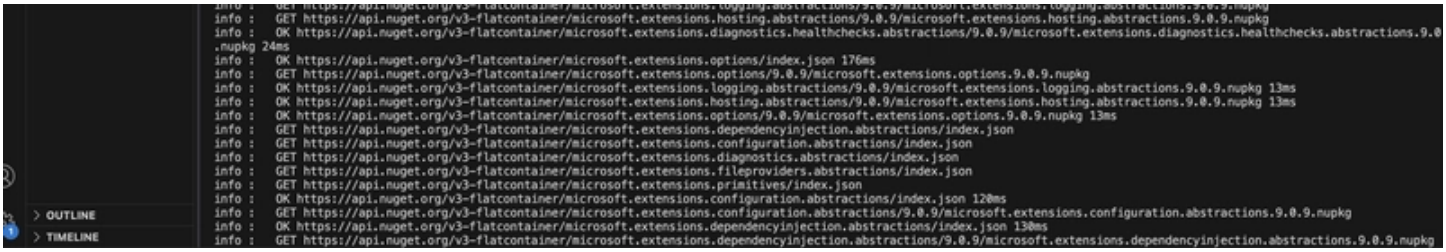


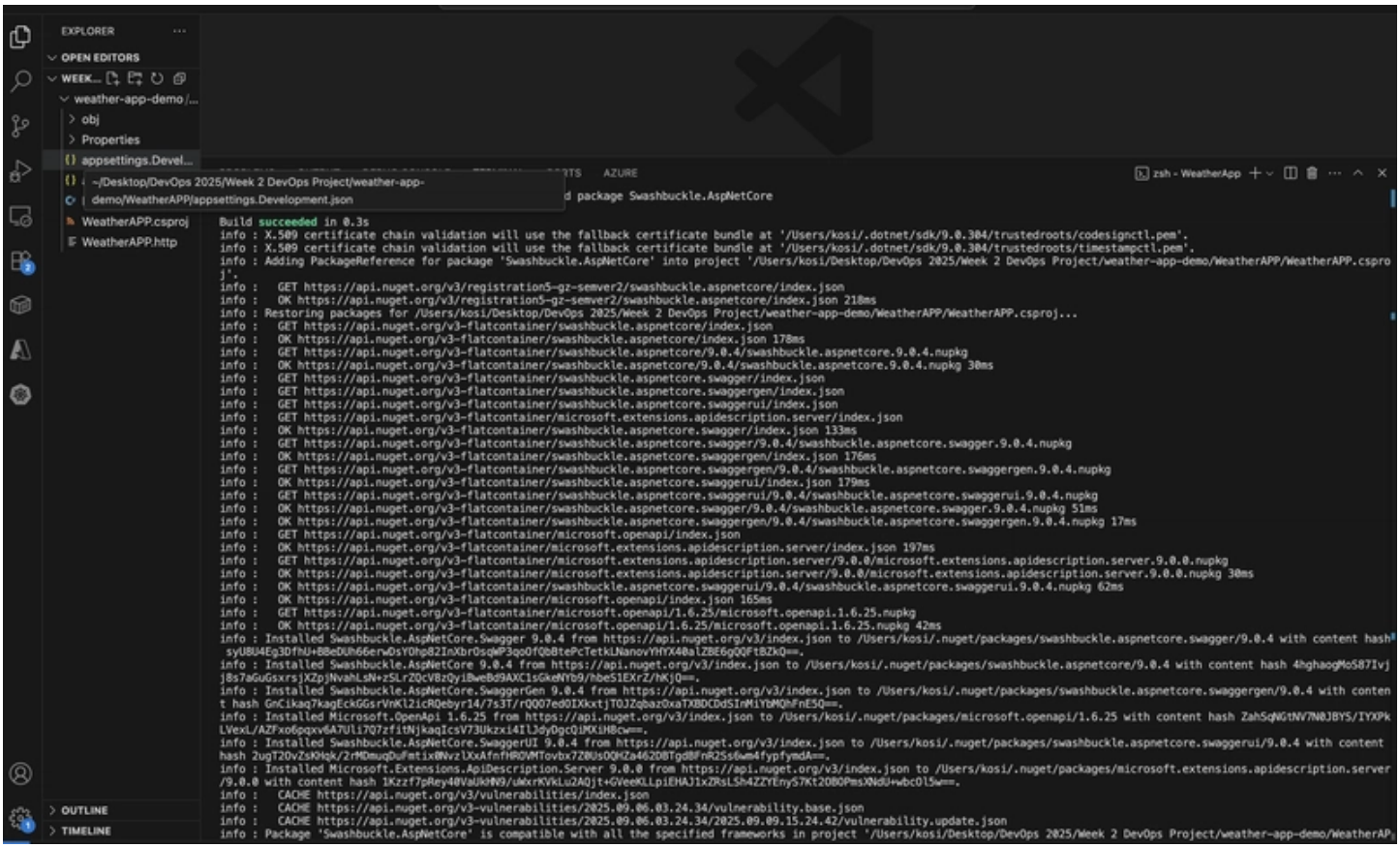
1.3 Add Dependencies

Install the required packages:

```
dotnet add package Microsoft.Extensions.Diagnostics.HealthChecks
dotnet add package Swashbuckle.AspNetCore
```







HealthChecks – adds a `/health` endpoint (important for Kubernetes).

Swashbuckle – provides Swagger UI for API docs.

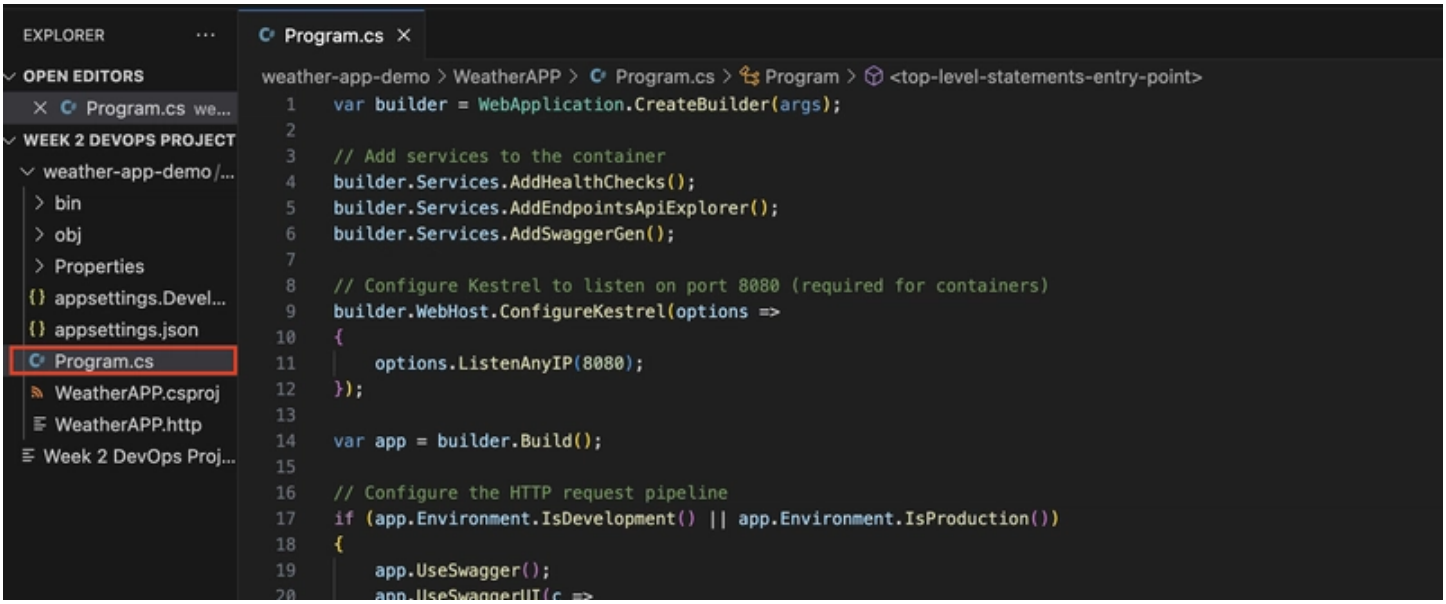
1.4 Write the Application Code

Replace the contents of Program.cs with the following:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container
builder.Services.AddHealthChecks();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Configure Kestrel to listen on port 8080 (required for containers)
```

```
builder.WebHost.ConfigureKestrel(options =>
{
    options.ListenAnyIP(8080);
});

var app = builder.Build();

// Configure the HTTP request pipeline
if (app.Environment.IsDevelopment() || app.Environment.IsProduction())
{
    app.UseSwagger();
    app.UseSwaggerUI(c =>
    {
        c.SwaggerEndpoint("/swagger/v1/swagger.json", "Weather API v1");
        c.RoutePrefix = "swagger";
    });
}

// Define API endpoints
app.MapGet("/", () => new
{
    Message = "Welcome to the Weather App!",
    Version = "1.0.0",
    Environment = app.Environment.EnvironmentName,
    Timestamp = DateTime.UtcNow
})
.WithName("GetWelcome")
.WithTags("General");

app.MapGet("/weather", () =>
{
    var summaries = new[] {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild",
        "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
    };

    var forecast = Enumerable.Range(1, 5).Select(index => new
```

```
    {
        Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
        TemperatureC = Random.Shared.Next(-20, 55),
        TemperatureF = 0,
        Summary = summaries[Random.Shared.Next(summaries.Length)]
    })
    .Select(temp => new
    {
        temp.Date,
        temp.TemperatureC,
        TemperatureF = 32 + (int)(temp.TemperatureC / 0.5556),
        temp.Summary
    });

    return forecast;
})
.WithName("GetWeatherForecast")
.WithTags("Weather");

// Health check endpoint (required for Kubernetes)
app.MapHealthChecks("/health")
.WithTags("Health");

app.Run();
```



```
    {
        c.SwaggerEndpoint("/swagger/v1/swagger.json", "Weather API v1");
        c.RoutePrefix = "swagger";
    });
}

// Define API endpoints
app.MapGet("/", () => new
{
    Message = "Welcome to the Weather App!",
    Version = "1.0.0",
    Environment = app.Environment.EnvironmentName,
    Timestamp = DateTime.UtcNow
})
.WithName("GetWelcome")
.WithTags("General");

app.MapGet("/weather", () =>
{
    var summaries = new[] {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild",
```

1.5 Run and Test Locally

Start the application:

`dotnet run`

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE

kosi@Kosisochukwus-MacBook-Pro WeatherApp % dotnet run
Using launch settings from /Users/kosi/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/WeatherAPP/Properties/launchSettings.json...
Building...
warn: Microsoft.AspNetCore.Server.Kestrel[0]
      Overriding address(es) 'http://localhost:5085'. Binding to endpoints defined via IConfiguration and/or UseKestrel() instead.
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://[::]:8080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /Users/kosi/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/WeatherAPP
```

Now, test the endpoints in your browser:

`http://localhost:8080/` → Welcome message

Pretty print ☐

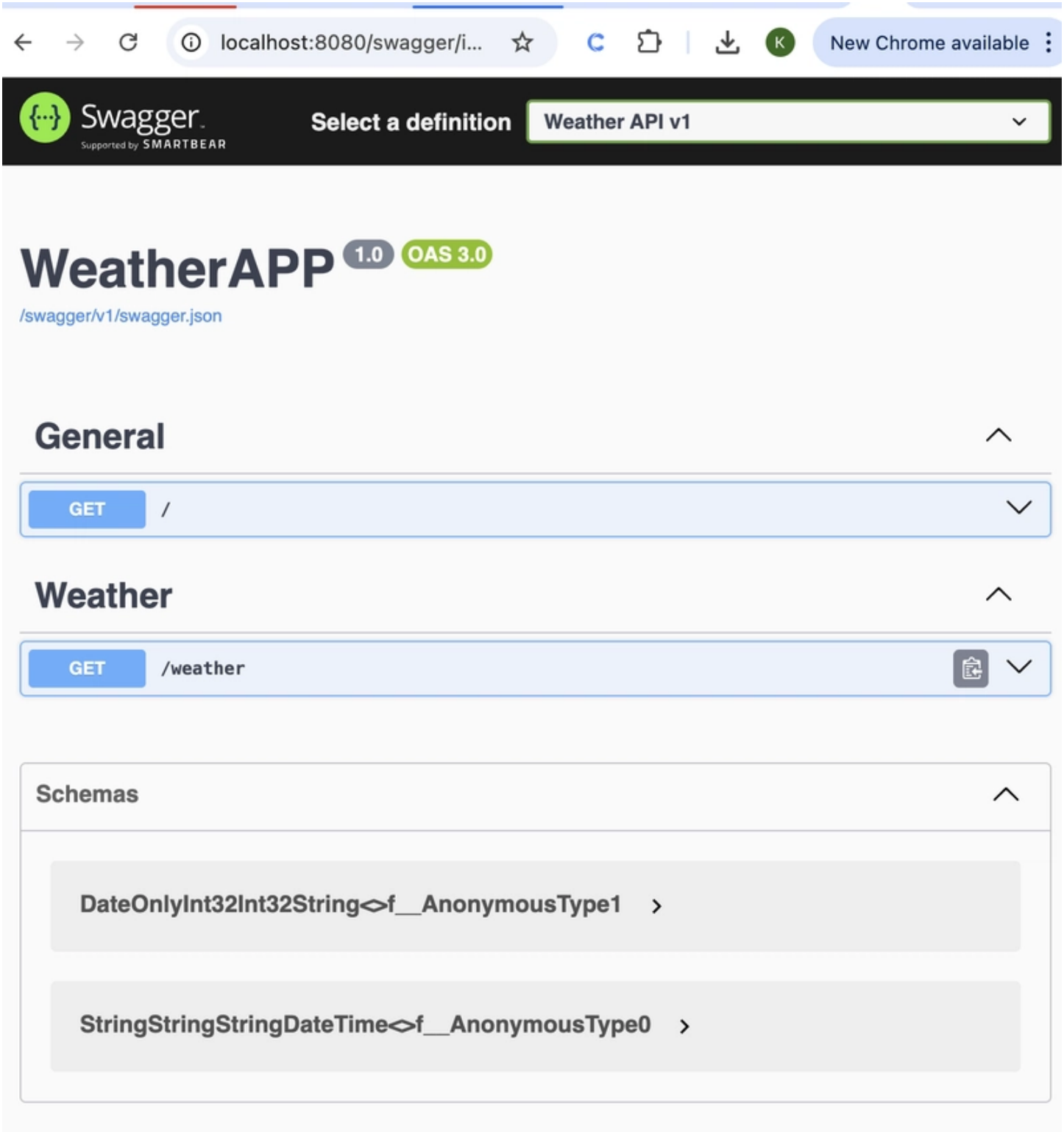

```
{"message":"Welcome to the Weather
App!","version":"1.0.0","environment":"Development","timestamp":"2025-09-09T18:41:51.83651Z"}
```


`http://localhost:8080/weather` → Weather forecast

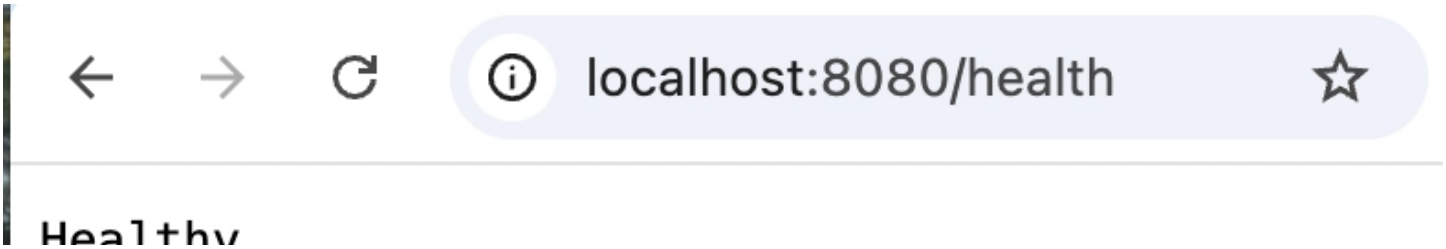
Pretty print ☐


```
[{"date":"2025-09-10","temperatureC":8,"temperatureF":46,"summary":"Scorching"},{"date":"2025-
09-11","temperatureC":22,"temperatureF":71,"summary":"Freezing"},{"date":"2025-09-
12","temperatureC":53,"temperatureF":127,"summary":"Freezing"},{"date":"2025-09-
13","temperatureC":43,"temperatureF":109,"summary":"Sweltering"},{"date":"2025-09-
14","temperatureC":23,"temperatureF":73,"summary":"Mild"}]
```

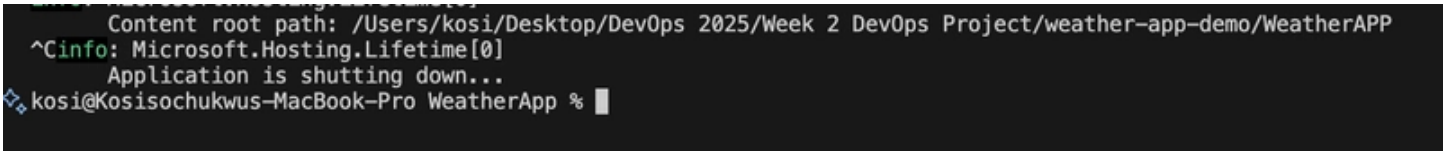
`http://localhost:8080/swagger` → API docs



`http://localhost:8080/health` → Health check endpoint



Press `Ctrl + C` to stop the app.



At this point, you have a working .NET API that's container-ready. Next, we willl Dockerize the app so it can run inside a container.

Step 2: Containerize Your Application

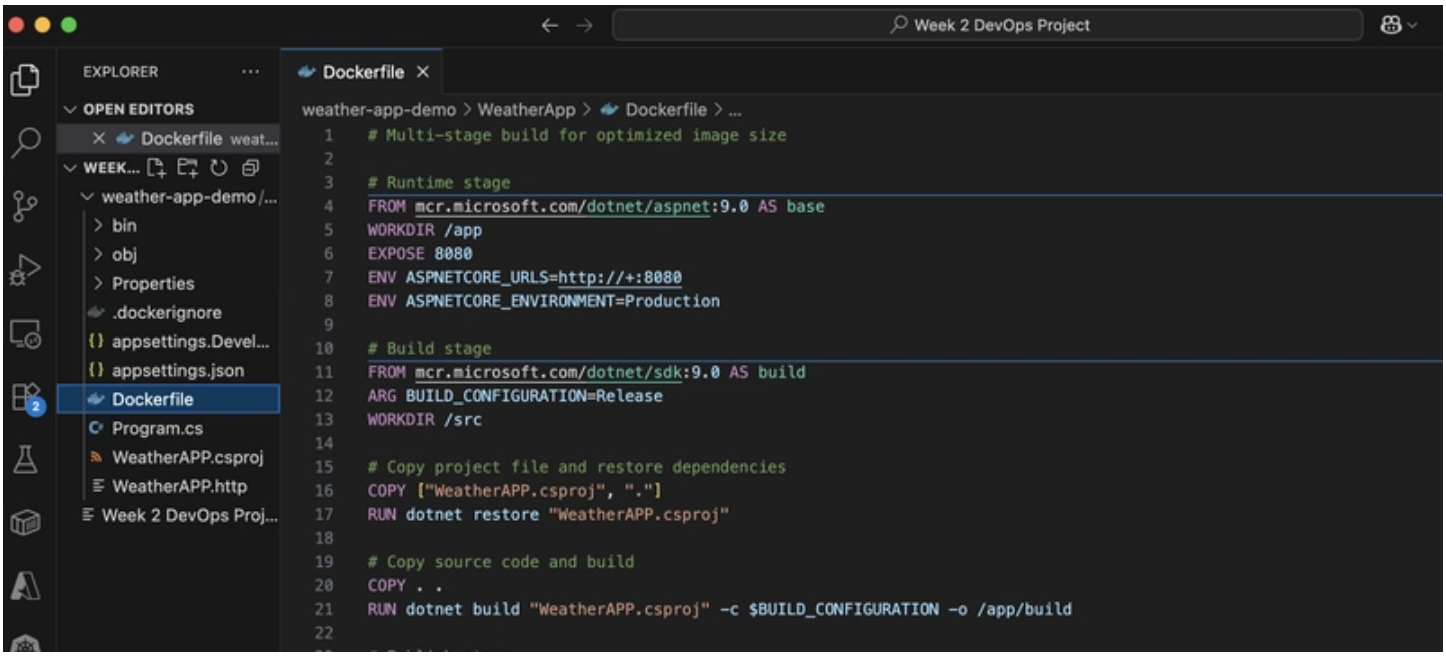
Now that the app runs locally, let's package it into a Docker container so it can run consistently anywhere.

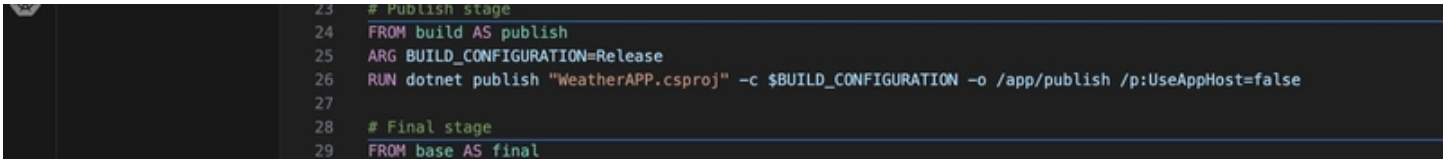
2.1 Create a Dockerfile

Inside the WeatherApp folder, create a file named Dockerfile (no extension) and add the following:

```
# Multi-stage build for optimized image size

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS base
WORKDIR /app
EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080
ENV ASPNETCORE_ENVIRONMENT=Production

# Build stage
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src
```

```
# Copy project file and restore dependencies
COPY ["WeatherApp/WeatherApp.csproj", "."]
RUN dotnet restore "WeatherApp.csproj"

# Copy source code and build
COPY . .
WORKDIR /src
COPY WeatherApp/ ./WeatherApp/
WORKDIR /src/WeatherApp
RUN dotnet restore "WeatherApp.csproj"
RUN dotnet build "WeatherApp.csproj" -c $BUILD_CONFIGURATION -o /app/build

# Publish stage
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "WeatherApp.csproj" -c $BUILD_CONFIGURATION -o /app/publish /
p:UseAppHost=false

# Final stage
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "WeatherApp.dll"]
```





```
# Publish stage
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "WeatherAPP.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

# Final stage
FROM base AS final
```

This uses a multi-stage build to keep the final image lightweight:

Build stage compiles your app.

Publish stage outputs optimized binaries.

Final stage only contains what's needed to run.

2.2 Create a .dockerignore File

Create a .dockerignore file in the root of your project to prevent unnecessary files from being copied into the Docker image:

```
# Build outputs
bin/
obj/
out/

# IDE files
.vs/
.vscode/
*.user
*.suo

# OS files
.DS_Store
Thumbs.db

# Git
.git/
.gitignore

# Documentation
README.md
*.md

# Docker
```

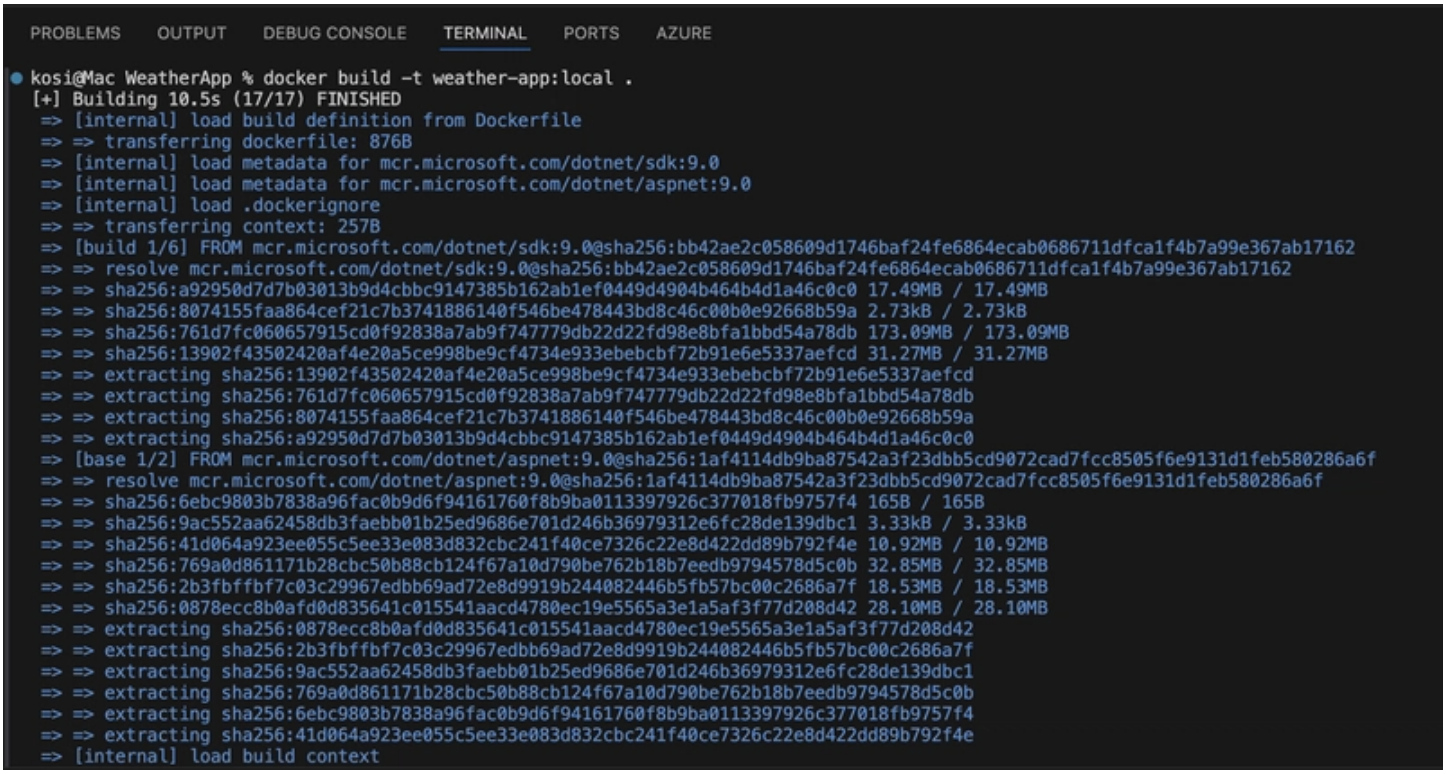
```
Dockerfile*
.dockerignore

# Logs
*.log
logs/
```

This keeps your Docker image clean and small.

2.3 Build and Test the Container

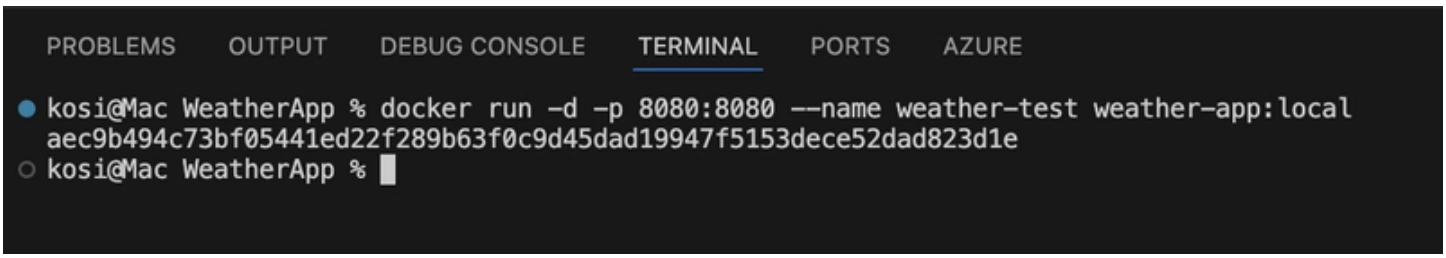
Build the Docker Image

`docker build -t weather-app:local .`



Run the Container

`docker run -d -p 8080:8080 --name weather-test weather-app:local`

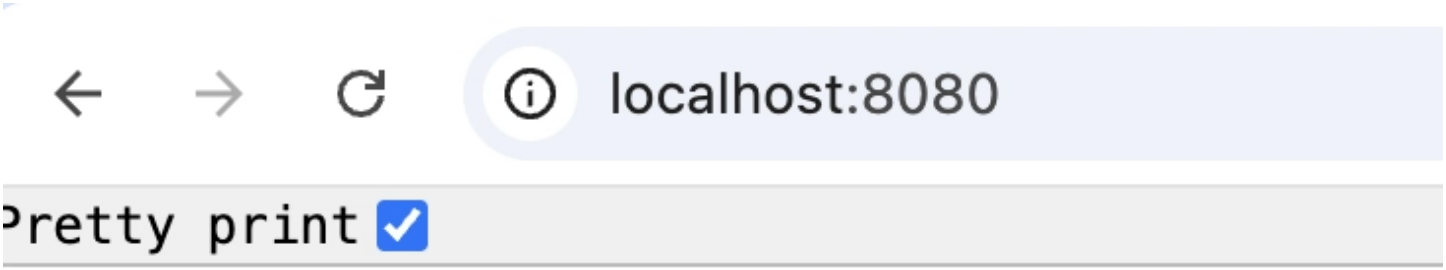


Test the App Inside Docker

```
curl http://localhost:8080/
curl http://localhost:8080/weather
```

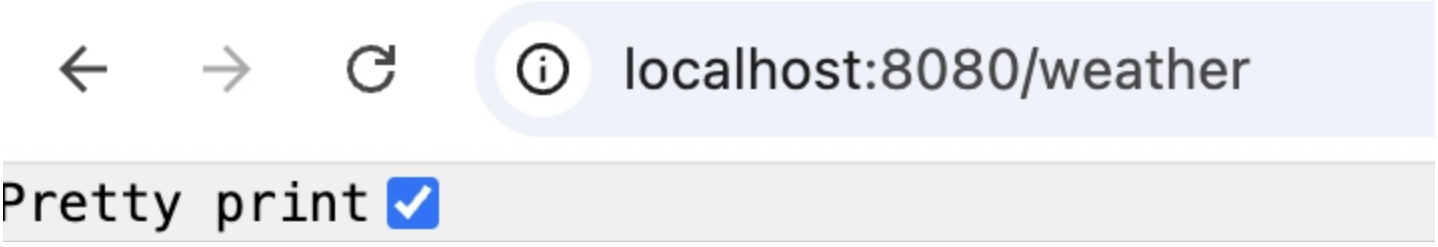
Or just visit in your browser:

```
http://localhost:8080/
```




```
{
  "message": "Welcome to the Weather App!",
  "version": "1.0.0",
  "environment": "Production",
  "timestamp": "2025-09-12T12:34:32.788382Z"
}
```


http://localhost:8080/weather

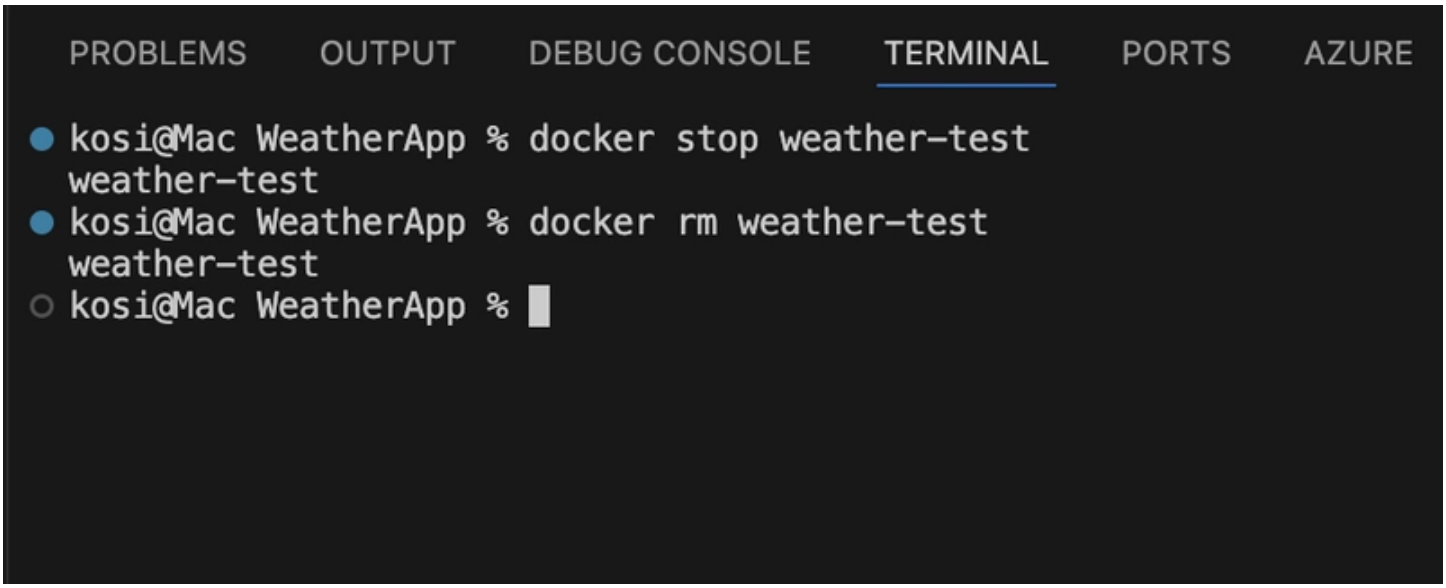


```
[
  {
    "date": "2025-09-13",
    "temperatureC": 43,
    "temperatureF": 109,
    "summary": "Scorching"
  },
  {
    "date": "2025-09-14",
    "temperatureC": -14,
    "temperatureF": 7,
    "summary": "Cool"
  },
  {
    "date": "2025-09-15",
    "temperatureC": 14,
    "temperatureF": 57,
    "summary": "Warm"
  },
  {
    "date": "2025-09-16",
    "temperatureC": 36,
    "temperatureF": 96,
    "summary": "Freezing"
  },
  {
    "date": "2025-09-17",
    "temperatureC": 48,
    "temperatureF": 118,
    "summary": "Freezing"
  }
```

]

Stop & Remove the Container

```
docker stop weather-test
docker rm weather-test
```



At this point, your app is running inside a container. Next, we'll push the container to a registry and deploy it to Azure Kubernetes Service (AKS).

Step 3: Set Up Azure Infrastructure

Now that your app runs inside a container, it's time to deploy it to the cloud. We will use Azure Kubernetes Service (AKS), along with Azure Container Registry (ACR) to store and distribute your Docker images.

3.1 Authenticate with Azure

Log in to your Azure account:

```
az login
```

This opens your browser for sign-in.

3.2 Select Your Subscription

List your available subscriptions:

```
az account list --output table
```

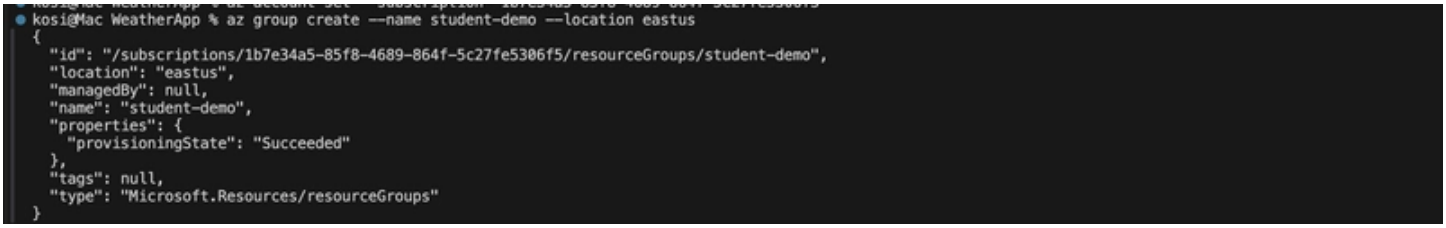
Set the subscription you want to use:

```
az account set --subscription "Your-Subscription-Name"
```

3.3 Create a Resource Group

A resource group is a logical container for all your Azure resources:

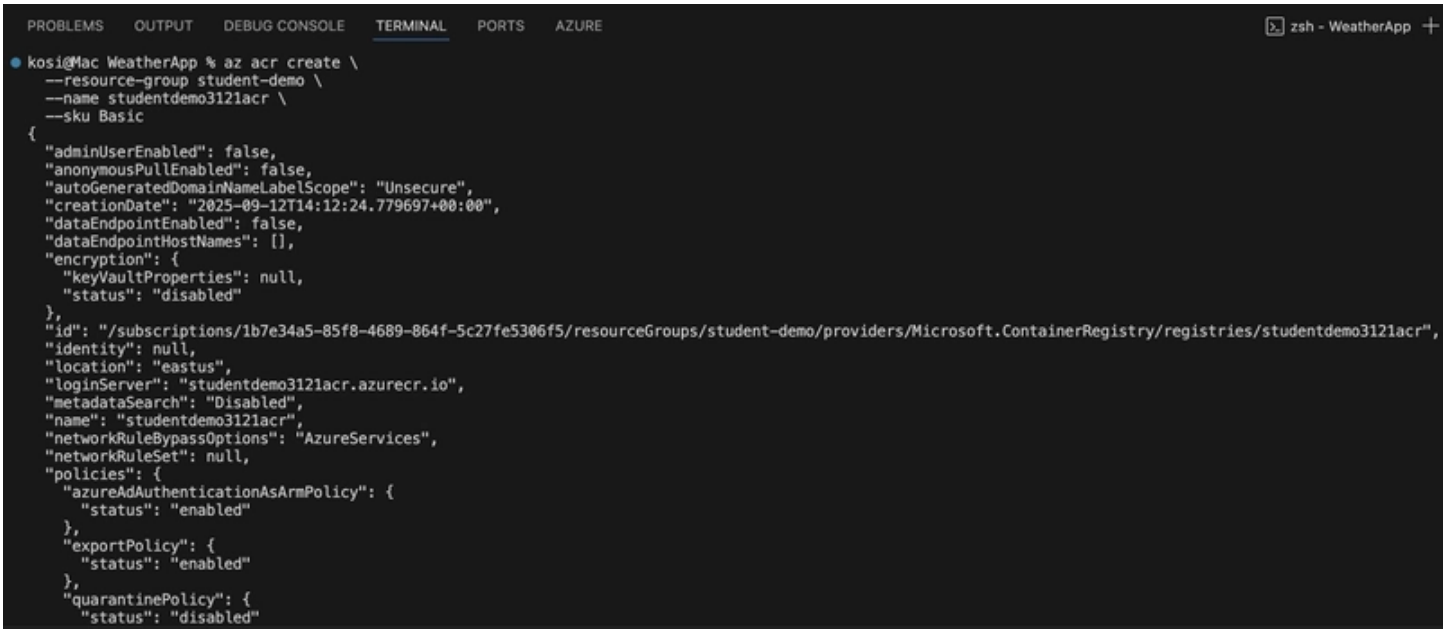
```
az group create --name student-demo --location eastus
```



3.4 Create a Container Registry (ACR)

We will use ACR to store our Docker images. Make sure the name is unique (append initials/year if needed):

```
az acr create \
--resource-group student-demo \
--name studentdemo3121acr \
--sku Basic
```



Note if you have an error while creating RG for your container registry, use this code below to register

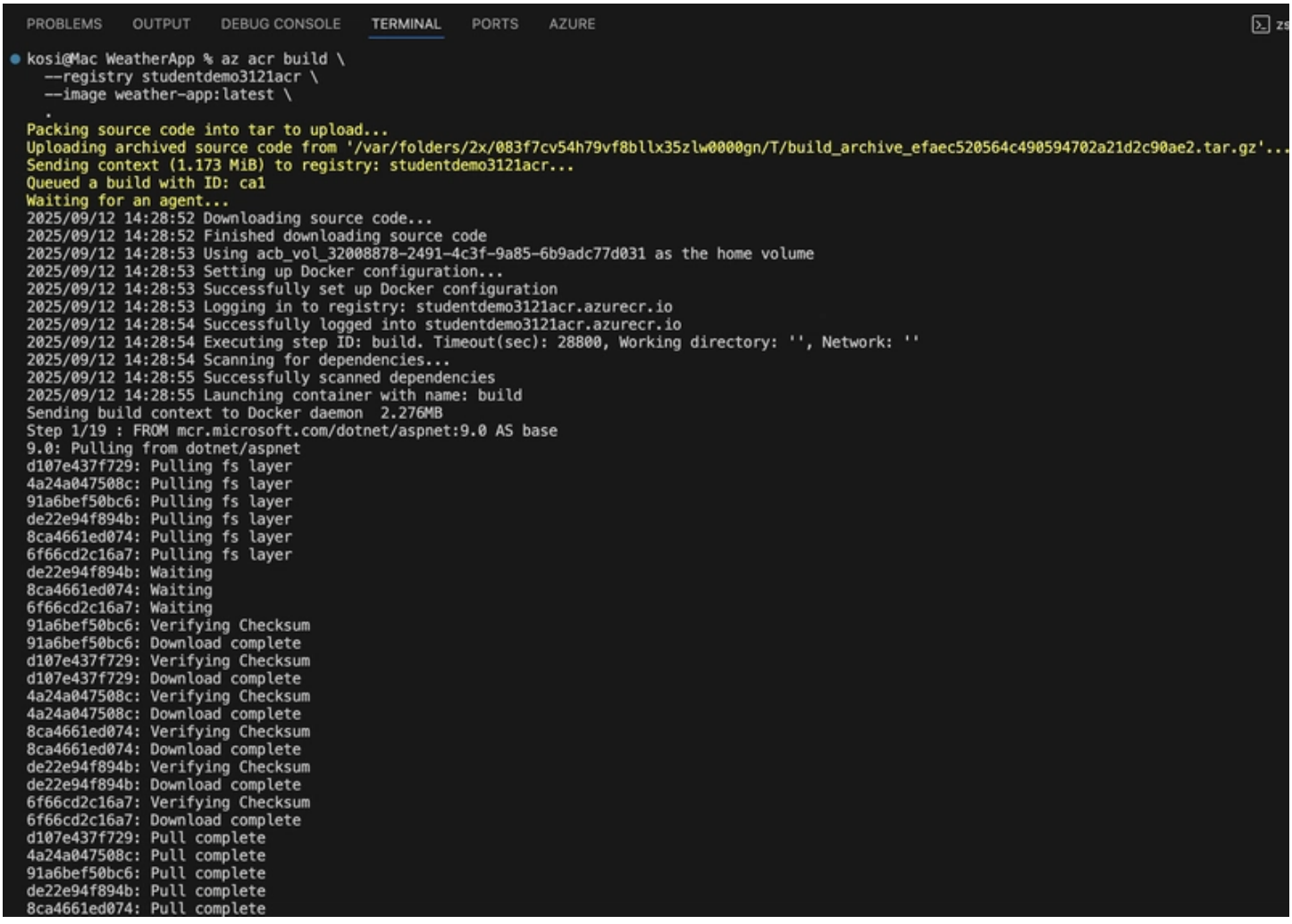
namespace on your container registry: `az provider register --namespace Microsoft.ContainerRegistry`

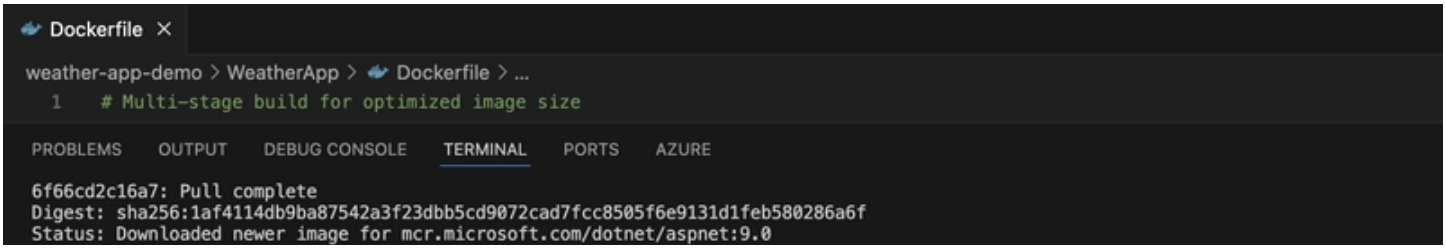
3.5 Build & Push the Image to ACR

Instead of building locally and pushing manually, we will let Azure build and push the image for us using az acr build.

From the folder containing your Dockerfile:

```
az acr build \
--registry studentdemo3121acr \
--image weather-app:latest \
.
```





```
 ---> 42df88dcd080
Step 2/19 : WORKDIR /app
 ---> Running in a9223cf2ea8f
Removing intermediate container a9223cf2ea8f
 ---> f7ec24200974
Step 3/19 : EXPOSE 8080
 ---> Running in 228e70095741
Removing intermediate container 228e70095741
 ---> b92fd3b259be
Step 4/19 : ENV ASPNETCORE_URLS=http://+:8080
 ---> Running in 1e17743a3e6b
Removing intermediate container 1e17743a3e6b
 ---> e3c251d73215
Step 5/19 : ENV ASPNETCORE_ENVIRONMENT=Production
 ---> Running in 0d645ce69ca8
Removing intermediate container 0d645ce69ca8
 ---> 37cdd6954400
Step 6/19 : FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
9.0: Pulling from dotnet/sdk
d107e437f729: Already exists
4a24a047508c: Already exists
91a6bef50bc6: Already exists
de22e94f894b: Already exists
8ca4661ed074: Already exists
6f66cd2c16a7: Already exists
42a538ee5b89: Pulling fs layer
68aa7387efe8: Pulling fs layer
ea29a56e41db: Pulling fs layer
70a8d67e460e: Pulling fs layer
70a8d67e460e: Waiting
ea29a56e41db: Download complete
42a538ee5b89: Verifying Checksum
42a538ee5b89: Download complete
70a8d67e460e: Verifying Checksum
70a8d67e460e: Download complete
68aa7387efe8: Verifying Checksum
68aa7387efe8: Download complete
42a538ee5b89: Pull complete
68aa7387efe8: Pull complete
ea29a56e41db: Pull complete
70a8d67e460e: Pull complete
Digest: sha256:bb42ae2c058609d1746baf24fe6864ecab0686711dfca1f4b7a99e367ab17162
Status: Downloaded newer image for mcr.microsoft.com/dotnet/sdk:9.0
 ---> 1aeff3957680
Step 7/19 : ARG BUILD_CONFIGURATION=Release
 ---> Running in 8756b1f0065b
```

Dockerfile ×

weather-app-demo > WeatherApp > Dockerfile > ...

```
1   # Multi-stage build for optimized image size
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

```
Step 7/19 : ARG BUILD_CONFIGURATION=Release
 ---> Running in 8756b1f0065b
Removing intermediate container 8756b1f0065b
 ---> d4143e132aa8
Step 8/19 : WORKDIR /src
 ---> Running in cfec14156300
Removing intermediate container cfec14156300
 ---> 67b1823df5d4
Step 9/19 : COPY ["WeatherAPP.csproj", "."]
 ---> b5ee4b45fb48
Step 10/19 : RUN dotnet restore "WeatherAPP.csproj"
 ---> Running in 4877d63810e9
  Determining projects to restore...
  Restored /src/WeatherAPP.csproj (in 1.9 sec).
Removing intermediate container 4877d63810e9
 ---> 92f4da912e0a
Step 11/19 : COPY . .
 ---> 345e03757e48
Step 12/19 : RUN dotnet build "WeatherAPP.csproj" -c $BUILD_CONFIGURATION -o /app/build
 ---> Running in de89ff0ef441
  Determining projects to restore...
  Restored /src/WeatherAPP.csproj (in 250 ms).
  WeatherAPP -> /app/build/WeatherAPP.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:04.62
Removing intermediate container de89ff0ef441
 ---> 85c3720d48ac
Step 13/19 : FROM build AS publish
```

```
 ---> 85c3720d48ac
Step 14/19 : ARG BUILD_CONFIGURATION=Release
 ---> Running in 99ae39dc5c93
Removing intermediate container 99ae39dc5c93
 ---> 8a172099b0e3
Step 15/19 : RUN dotnet publish "WeatherAPP.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false
 ---> Running in e7dba66b0823
  Determining projects to restore...
  All projects are up-to-date for restore.
  WeatherAPP -> /src/bin/Release/net9.0/WeatherAPP.dll
  WeatherAPP -> /app/publish/
Removing intermediate container e7dba66b0823
 ---> 3f110a250ffc
Step 16/19 : FROM base AS final
 ---> 37cdd6954400
Step 17/19 : WORKDIR /app
 ---> Running in 4b646fd02473
```

Dockerfile ×

weather-app-demo > WeatherApp > Dockerfile > ...

```
1   # Multi-stage build for optimized image size
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

```
2025/09/12 14:29:32 Executing step ID: push. Timeout(sec): 3600, Working directory: '', Network: ''
2025/09/12 14:29:32 Pushing image: studentdemo3121acr.azurecr.io/weather-app:latest, attempt 1
The push refers to repository [studentdemo3121acr.azurecr.io/weather-app]
dd580f61dba3: Preparing
ab53bc7d5374: Preparing
9b4f978c8fc6: Preparing
4289c63563cf: Preparing
3a3dbab1a8ca: Preparing
c0691992e8c0: Preparing
6dfd5c4c2433: Preparing
36f5f951f60a: Preparing
c0691992e8c0: Waiting
6dfd5c4c2433: Waiting
36f5f951f60a: Waiting
ab53bc7d5374: Pushed
9b4f978c8fc6: Pushed
4289c63563cf: Pushed
c0691992e8c0: Pushed
dd580f61dba3: Pushed
3a3dbab1a8ca: Pushed
6dfd5c4c2433: Pushed
36f5f951f60a: Pushed
latest: digest: sha256:8c851e7980d33de792af7cd87e1f6de8a9d592fb803c5d478a7980b70ae76dc3 size: 1997
2025/09/12 14:29:41 Successfully pushed image: studentdemo3121acr.azurecr.io/weather-app:latest
2025/09/12 14:29:41 Step ID: build marked as successful (elapsed time in seconds: 37.329014)
2025/09/12 14:29:41 Populating digests for step ID: build...
2025/09/12 14:29:42 Successfully populated digests for step ID: build
2025/09/12 14:29:42 Step ID: push marked as successful (elapsed time in seconds: 9.242887)
2025/09/12 14:29:42 The following dependencies were found:
2025/09/12 14:29:42
- image:
    registry: studentdemo3121acr.azurecr.io
    repository: weather-app
    tag: latest
    digest: sha256:8c851e7980d33de792af7cd87e1f6de8a9d592fb803c5d478a7980b70ae76dc3
  runtime-dependency:
    registry: mcr.microsoft.com
    repository: dotnet/aspnet
    tag: "9.0"
    digest: sha256:1af4114db9ba87542a3f23dbb5cd9072cad7fcc8505f6e9131d1feb580286a6f
  buildtime-dependency:
  - registry: mcr.microsoft.com
    repository: dotnet/sdk
    tag: "9.0"
    digest: sha256:bb42ae2c058609d1746baf24fe6864ecab0686711dfca1f4b7a99e367ab17162
  git: {}

Run ID: ca1 was successful after 51s
```

Note: Below is what the code means:

registry studentdemo3121acr → Your ACR name

image weather-app:latest → Tags the image inside ACR

Uses the Dockerfile in the current directory

Note: This avoids compatibility issues since the build runs directly in Azure.

3.6 Create an AKS Cluster with ACR Integration

Now let's create the Kubernetes cluster where our app will run:

```
az aks create \
  --resource-group student-demo \
  --name student-aks-cluster \
  --node-count 1 \
  --node-vm-size Standard_B2s \
  --attach-acr studentdemo2024acr \
  --enable-managed-identity \
  --generate-ssh-keys
```

Dockerfile

weather-app-demo > WeatherApp > Dockerfile > ...

```
1  # Multi-stage build for optimized image size
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

```
kosi@Mac WeatherApp % az aks create \
  --resource-group student-demo \
  --name student-aks-cluster \
  --node-count 1 \
  --node-vm-size Standard_B2s \
  --attach-acr studentdemo3121acr \
  --enable-managed-identity \
  --generate-ssh-keys
AAD role propagation done[##############################################]  100.0000%{
  "aadProfile": null,
  "addonProfiles": null,
  "agentPoolProfiles": [
    {
      "availabilityZones": null,
      "capacityReservationGroupId": null,
      "count": 1,
      "creationData": null,
      "currentOrchestratorVersion": "1.32.6",
      "eTag": null,
      "enableAutoScaling": false,
      "enableEncryptionAtHost": false,
      "enableFips": false,
      "enableNodePublicIp": false,
      "enableUltraSsd": false,
      "gatewayProfile": null,
      "gpuInstanceProfile": null,
      "gpuProfile": null,
      "hostGroupId": null,
      "kubeletConfig": null,
      "kubeletDiskType": "OS",
      "linuxOsConfig": null,
      "maxCount": null,
      "maxPods": 250,
      "messageOfTheDay": null,
      "minCount": null,
      "mode": "System",
      "name": "nodepool1",
      "networkProfile": null,
      "nodeImageVersion": "AKSUbuntu-2204gen2containerd-202508.20.1",
      "nodeLabels": null,
      "nodePublicIpPrefixId": null,
      "nodeTaints": null,
      "orchestratorVersion": "1.32",
      "osDiskSizeGb": 128,
      "osDiskType": "Managed",
      "osSku": "Ubuntu",
      "osType": "Linux",
      "podIpAllocationMode": null,
      "podSubnetId": null,
```

Dockerfile

weather-app-demo > WeatherApp > Dockerfile > ...

```
1   # Multi-stage build for optimized image size
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE


```
      "powerState": {
        "code": "Running"
      },
      "provisioningState": "Succeeded",
      "proximityPlacementGroupId": null,
      "scaleDownMode": "Delete",
      "scaleSetEvictionPolicy": null,
      "scaleSetPriority": null,
      "securityProfile": {
        "enableSecureBoot": false,
        "enableVtpm": false
      },
      "spotMaxPrice": null,
      "status": null,
      "tags": null,
      "type": "VirtualMachineScaleSets",
      "upgradeSettings": {
        "drainTimeoutInMinutes": null,
        "maxSurge": "10%",
        "maxUnavailable": "0",
        "nodeSoakDurationInMinutes": null,
        "undrainableNodeBehavior": null
      },
      "virtualMachineNodesStatus": null,
      "virtualMachinesProfile": null,
      "vmSize": "Standard_B2s",
      "vnetSubnetId": null,
      "windowsProfile": null,
      "workloadRuntime": null
    }
  ],
  "aiToolchainOperatorProfile": null,
  "apiServerAccessProfile": null,
  "autoScalerProfile": null,
  "autoUpgradeProfile": {
    "nodeOsUpgradeChannel": "NodeImage",
    "upgradeChannel": null
  },
  "azureMonitorProfile": null,
  "azurePortalFqdn": "student-ak-student-demo-1b7e34-yi6bn4ek.portal.hcp.eastus.azmk8s.io",
  "bootstrapProfile": {
    "artifactSource": "Direct",
    "containerRegistryId": null
  },
  "currentKubernetesVersion": "1.32.6",
  "disableLocalAccounts": false,
  "diskEncryptionSetId": null,
  "dnsPrefix": "student-ak-student-demo-1b7e34",
  "eTag": null,
```

Dockerfile

weather-app-demo > WeatherApp > Dockerfile > ...

```
1  # Multi-stage build for optimized image size
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

zsh - WeatherApp


```
  "dnsPrefix": "student-ak-student-demo-1b7e34",
  "eTag": null,
  "enableRbac": true,
  "extendedLocation": null,
  "fqdn": "student-ak-student-demo-1b7e34-yi6bn4ek.hcp.eastus.azmk8s.io",
  "fqdnSubdomain": null,
  "httpProxyConfig": null,
  "id": "/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourcegroups/student-demo/providers/Microsoft.ContainerService/managedClusters/student-aks-cluster",
  "identity": {
    "delegatedResources": null,
    "principalId": "29b371dd-3ba9-4829-b34d-bf6765b7fbda",
    "tenantId": "ec32aec3-a06f-441b-8910-226adb4d58de",
    "type": "SystemAssigned",
    "userAssignedIdentities": null
  },
  "identityProfile": {
    "kubeletidentity": {
      "clientId": "66cd14ea-24a6-45b0-9e4f-1a4809cb35dc",
      "objectId": "0a9063ef-863e-4b3c-8d5c-4a9c938e6385",
      "resourceId": "/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourcegroups/MC_student-demo_student-aks-cluster_eastus/providers/Microsoft.ManagedIdentity/user
AssignedIdentities/student-aks-cluster-agentpool"
    }
  },
  "ingressProfile": null,
  "kind": "Base",
  "kubernetesVersion": "1.32",
  "linuxProfile": {
    "adminUsername": "azureuser",
    "ssh": {
      "publicKeys": [
        {
          "keyData": "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQC3Pabzp0g14E4pS/C0nGdXtSF+ZKFpWC1xyz9A7ZRfgfowPdfQtUNMCTd84o+AoGQHyK/CoanHGHBLgFyJ4aJi0rQb8u3Xk2stX4bzFgPZlIKHn
SQR14N5x+qdwuc62OALVLfuhAE8uflbgEuENpaYNz/Q5yvtiZ2zd1otZ2iiw6tlZEjEfvKEUfSRkkSjhKgc639NuhZJYsAbV6+XtoO6dyKbdkxZhDLDI1/xrsSSW8BzCTvaD7fcIFydOQyajim4u50WHNb4hRvO0p5SjtxJdSJ6
Dg9AxNgHz0dEzCO5hzWjzuOhWVPQoa8c2mVaXJkSOQyFdnNe+3QYByXkCsvr"
        }
      ]
    }
  },
  "location": "eastus",
  "maxAgentPools": 100,
  "metricsProfile": {
    "costAnalysis": {
      "enabled": false
    }
  },
  "name": "student-aks-cluster",
  "networkProfile": {
    "advancedNetworking": null,
    "dnsServiceIp": "10.0.0.10",
```


Dockerfile

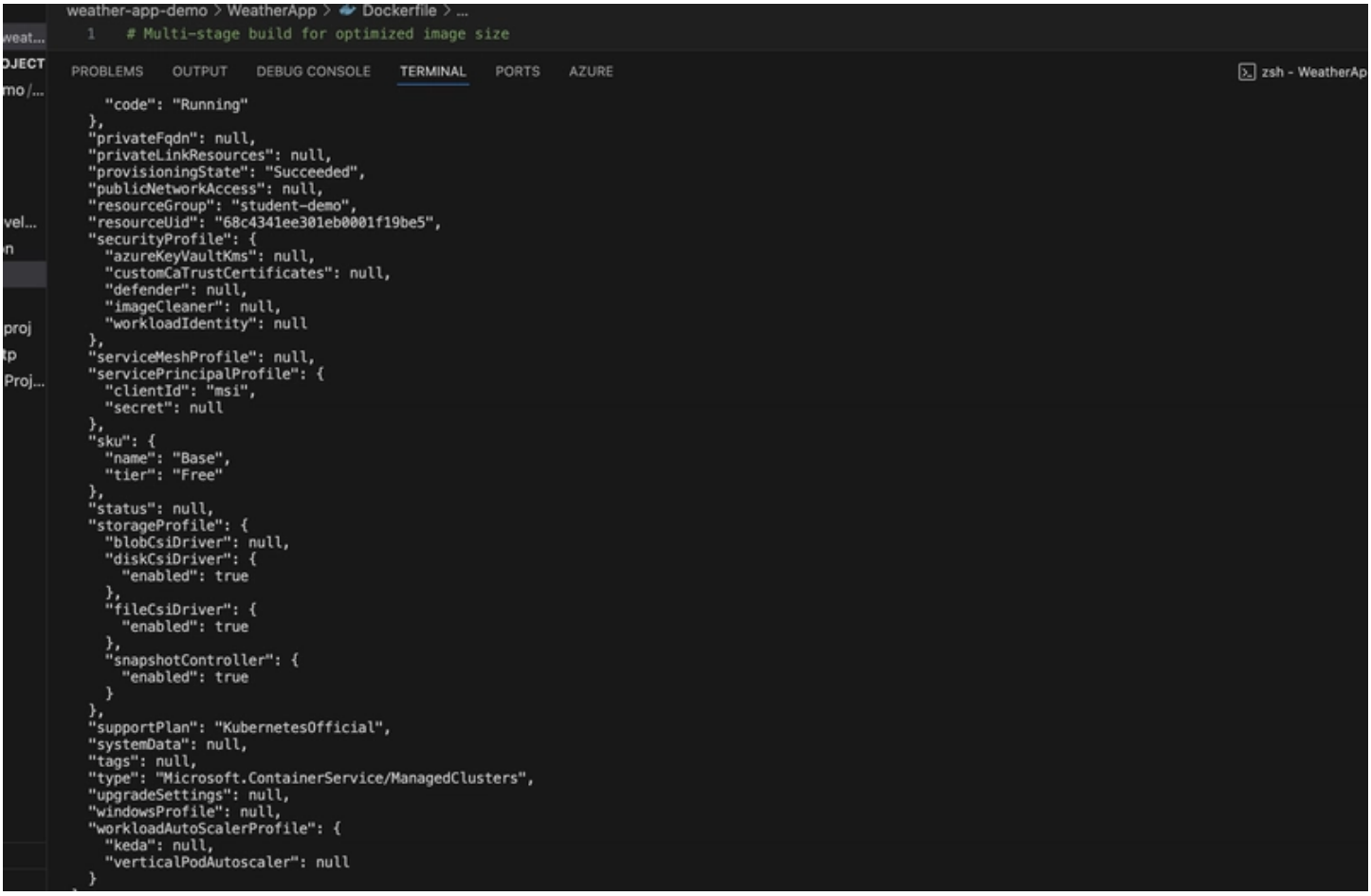
weather-app-demo > WeatherApp > Dockerfile > ...

```
1  # Multi-stage build for optimized image size
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

zsh - WeatherApp


```
    "dnsServiceIp": "10.0.0.10",
    "ipFamilies": [
      "IPv4"
    ],
    "loadBalancerProfile": {
      "allocatedOutboundPorts": null,
      "backendPoolType": "nodeIPConfiguration",
      "effectiveOutboundIPs": [
        {
          "id": "/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourceGroups/MC_student-demo_student-aks-cluster_eastus/providers/Microsoft.Network/publicIPAddresse
s/d26610fa-572c-469c-bee6-f836b4fc13ab",
          "resourceGroup": "MC_student-demo_student-aks-cluster_eastus"
        }
      ],
      "enableMultipleStandardLoadBalancers": null,
      "idleTimeoutInMinutes": null,
      "managedOutboundIPs": {
        "count": 1,
        "countIpv6": null
      },
      "outboundIPs": null,
      "outboundIpPrefixes": null
    },
    "loadBalancerSku": "standard",
    "natGatewayProfile": null,
    "networkDataplane": "azure",
    "networkMode": null,
    "networkPlugin": "azure",
    "networkPluginMode": "overlay",
    "networkPolicy": "none",
    "outboundType": "loadBalancer",
    "podCidr": "10.244.0.0/16",
    "podCidrs": [
      "10.244.0.0/16"
    ],
    "serviceCidr": "10.0.0.0/16",
    "serviceCidrs": [
      "10.0.0.0/16"
    ],
    "staticEgressGatewayProfile": null
  },
  "nodeProvisioningProfile": {
    "defaultNodePools": "Auto",
    "mode": "Manual"
  },
  "nodeResourceGroup": "MC_student-demo_student-aks-cluster_eastus",
  "nodeResourceGroupProfile": null,
  "oidcIssuerProfile": {
    "enabled": false,
```



What this does:

Creates a managed AKS cluster

Adds 1 VM node (2 vCPUs, 4 GB RAM)

Integrates ACR automatically (avoids ImagePullBackOff errors)

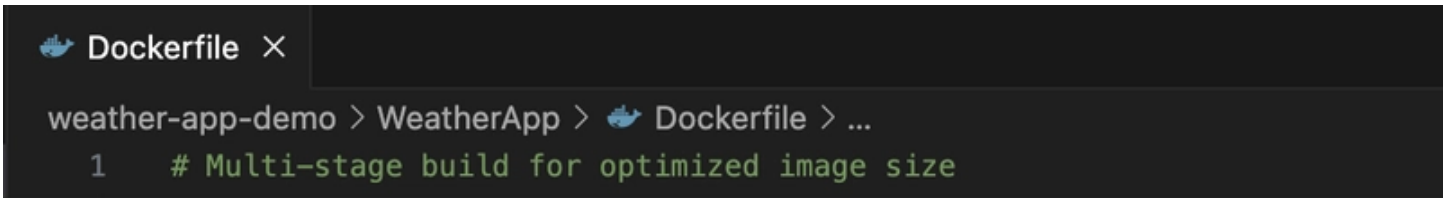
Generates SSH keys for secure access

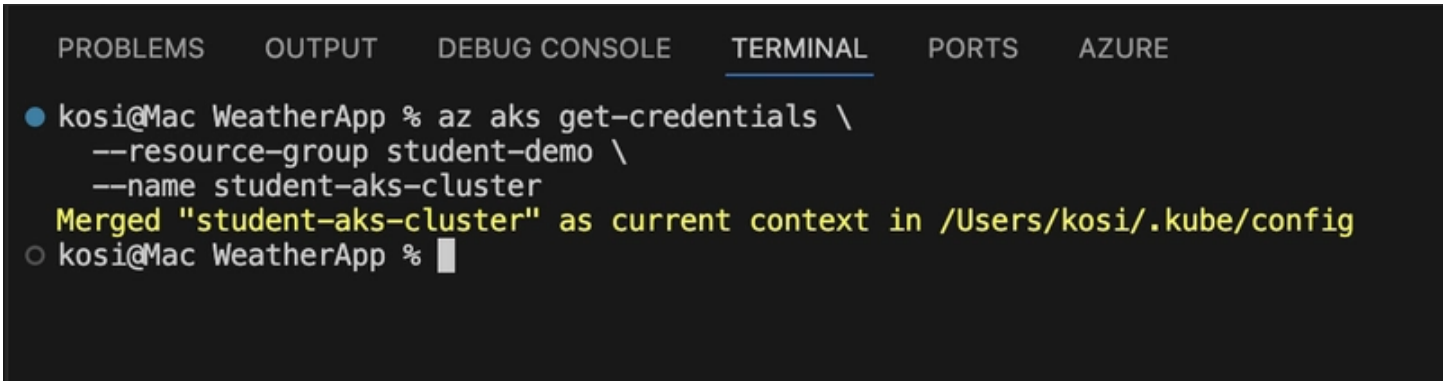
Now wait 5–10 minutes for Azure to provision everything (coffee break time ☕).

3.7 Connect to Your AKS Cluster

Once the cluster is ready, configure kubectl to connect:

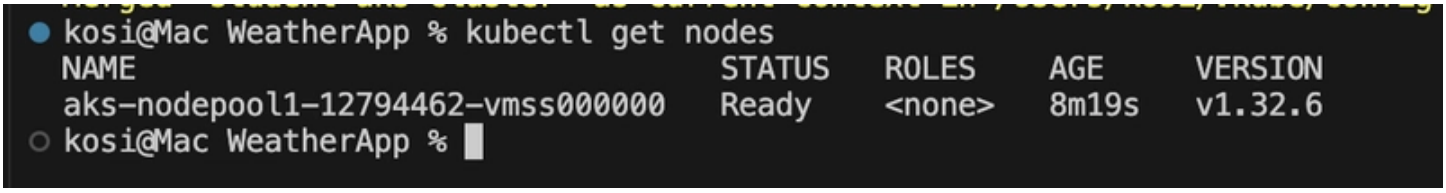
```
az aks get-credentials \
--resource-group student-demo \
--name student-aks-cluster
```





Verify the connection:

```
kubectl get nodes
```

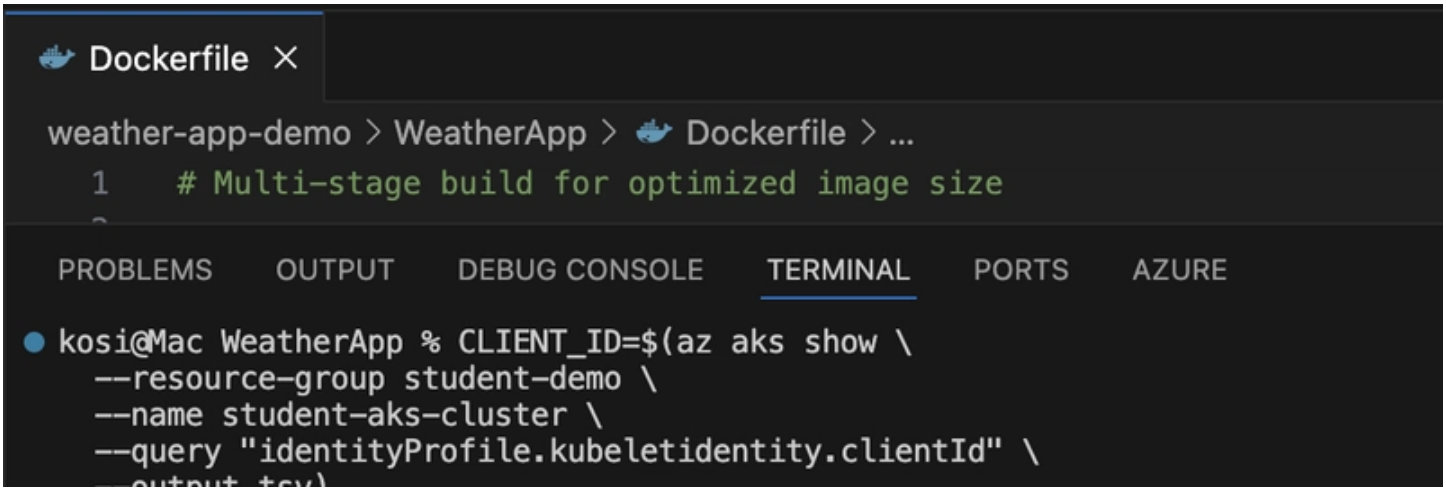


You should see your Kubernetes node(s) listed.

3.8 Verify ACR Integration (Optional)

Check that your AKS cluster can pull images from ACR:

```
CLIENT_ID=$(az aks show \
  --resource-group student-demo \
  --name student-aks-cluster \
  --query "identityProfile.kubeletidentity.clientId" \
  --output tsv)

echo "Kubelet Client ID: $CLIENT_ID"
```



```
--output tsv)
kosi@Mac WeatherApp % echo "Kubelet Client ID: $CLIENT_ID"
Kubelet Client ID: 66cd14ea-24a6-45b0-9e4f-1a4809cb35dc
kosi@Mac WeatherApp %
```

Now confirm the role assignment:

```
az role assignment list \
--assignee $CLIENT_ID \
--scope $(az acr show --name studentdemo3121acr --query id --output tsv) \
--output table
```

```
kosi@Mac WeatherApp % az role assignment list \
  --assignee $CLIENT_ID \
  --scope $(az acr show --name studentdemo3121acr --query id --output tsv) \
  --output table
Principal                             Role     Scope
------------------------------------  -------  ------------------------------------
----------------------
```

You should see an AcrPull role. If not, create it manually:

```
az role assignment create \
--assignee $CLIENT_ID \
--role AcrPull \
--scope $(az acr show --name studentdemo3121acr --query id --output tsv)
```

At this point, you have a fully provisioned Azure Kubernetes cluster and a container image stored in ACR. Next, we will deploy the app into Kubernetes.

Step 4: Create Kubernetes Configuration

Now that we have our AKS cluster and image in ACR, it's time to define the Kubernetes manifests that describe how our app should run in the cluster.

4.1 Set Up a Manifests Directory

Organize your Kubernetes YAML files inside a k8s folder:

`cd ..` to Go back to weather-app-demo root
`mkdir k8s`
`cd k8s`

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE
```

```
kosi@Mac WeatherApp % cd ..
kosi@Mac weather-app-demo % mkdir k8s
kosi@Mac weather-app-demo % cd k8s
kosi@Mac k8s %
```

4.2 Create the Deployment

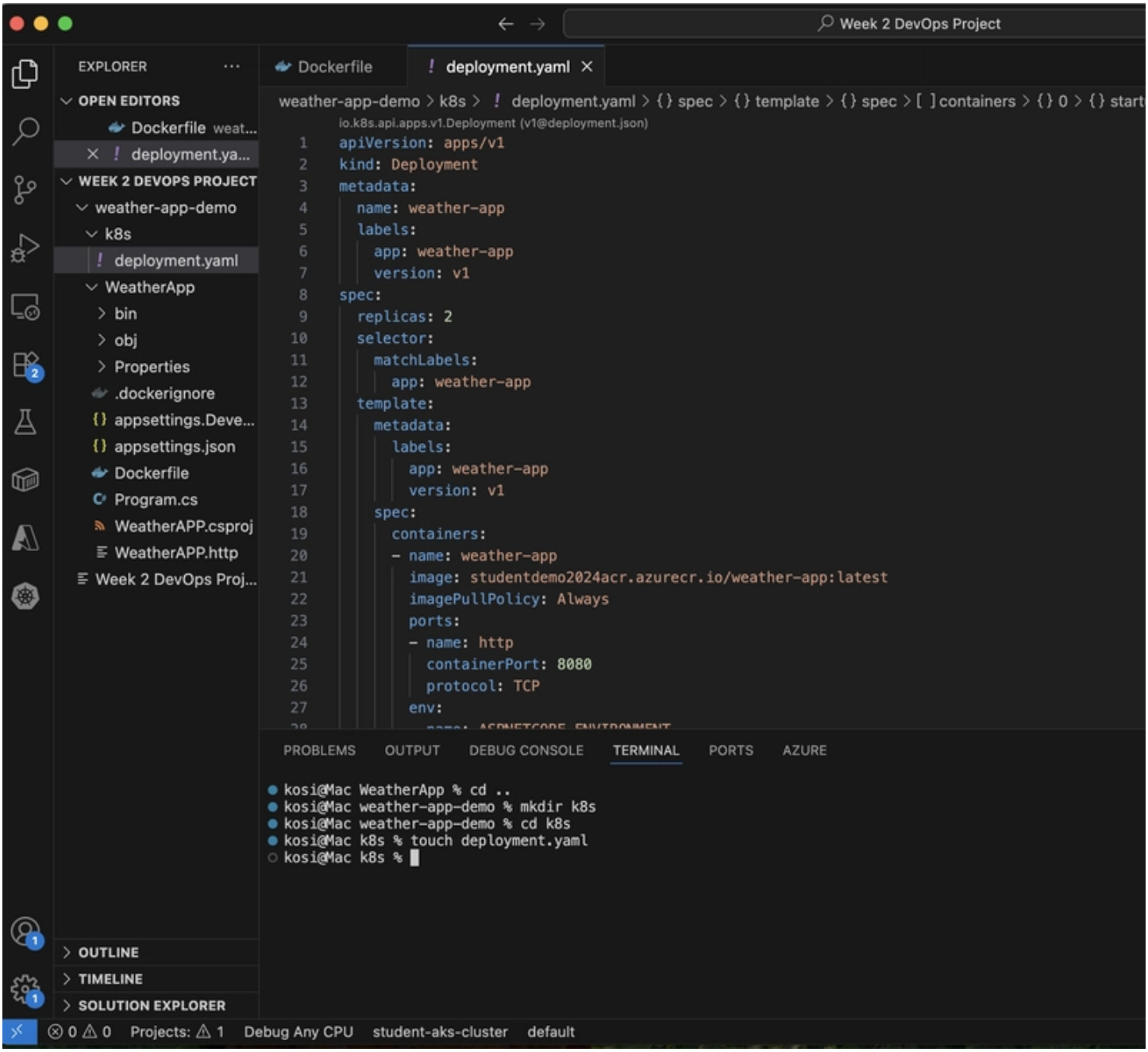
A Deployment ensures your app runs in Kubernetes with the desired number of replicas, health checks, and resource limits.

Create a file called deployment.yaml:

`touch .deployment.yaml` - This creates a file
Now copy and paste the code below into the `deployment.yaml` file created

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: weather-app
  labels:
    app: weather-app
    version: v1
spec:
  replicas: 2
  selector:
    matchLabels:
      app: weather-app
  template:
    metadata:
      labels:
        app: weather-app
        version: v1
    spec:
      containers:
      - name: weather-app
        image: studentdemo2024acr.azurecr.io/weather-app:latest
        imagePullPolicy: Always
        ports:
```

```
    - name: http
      containerPort: 8080
      protocol: TCP
    env:
    - name: ASPNETCORE_ENVIRONMENT
      value: "Production"
    resources:
      requests:
        memory: "128Mi"
        cpu: "100m"
      limits:
        memory: "512Mi"
        cpu: "500m"
    livenessProbe:
      httpGet:
        path: /health
        port: http
      initialDelaySeconds: 30
      periodSeconds: 10
      timeoutSeconds: 5
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /health
        port: http
      initialDelaySeconds: 10
      periodSeconds: 5
      timeoutSeconds: 3
      failureThreshold: 3
    startupProbe:
      httpGet:
        path: /health
        port: http
      initialDelaySeconds: 10
      periodSeconds: 5
      timeoutSeconds: 3
      failureThreshold: 30
```



Notes:

Uses 2 replicas for high availability.

Includes liveness, readiness, and startup probes to let Kubernetes know when the app is healthy.

No `imagePullSecrets` needed because AKS was created with `--attach-acr` to handle authentication automatically!

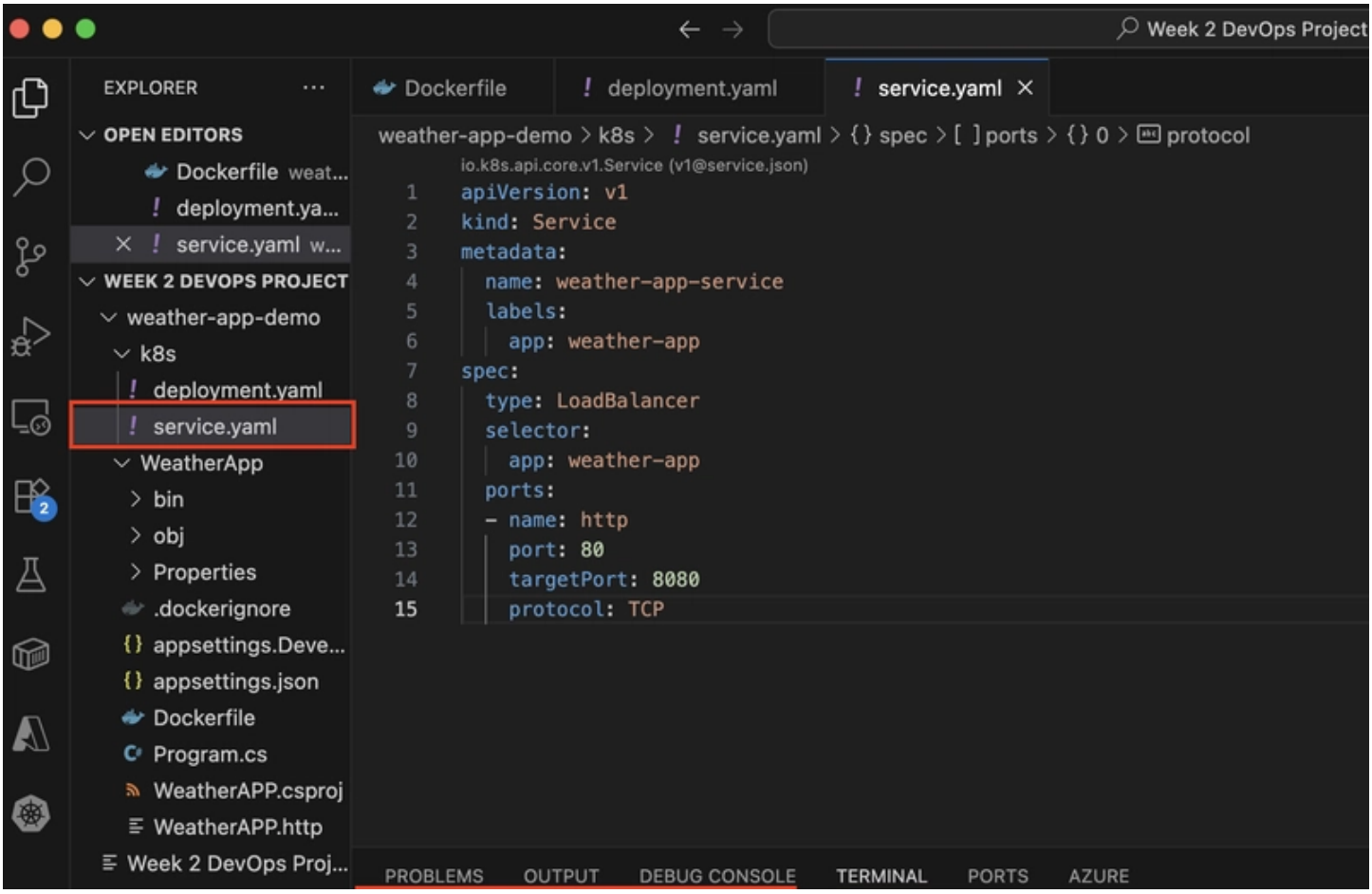
4.3 Create the Service

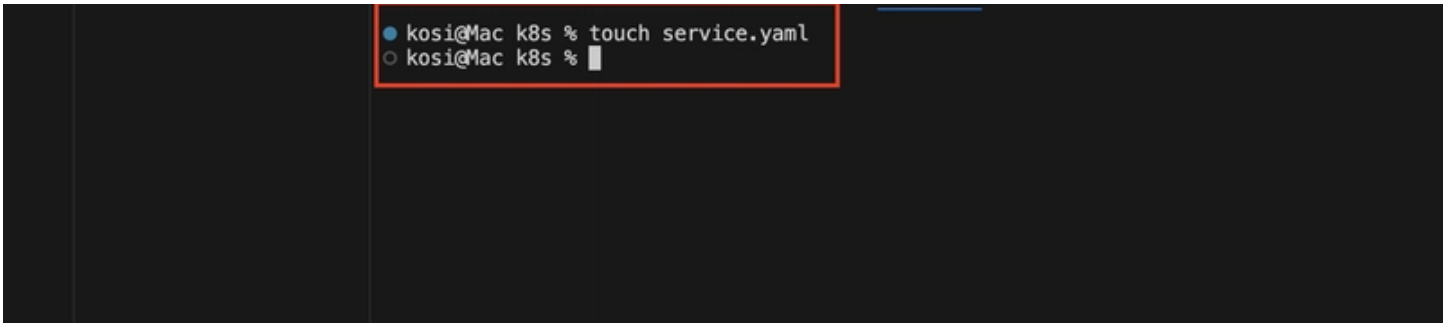
A Service exposes your app to the internet through a cloud load balancer.

Create a file called service.yaml:

Same process we used in 4.2

```
apiVersion: v1
kind: Service
metadata:
  name: weather-app-service
  labels:
    app: weather-app
spec:
  type: LoadBalancer
  selector:
    app: weather-app
  ports:
  - name: http
    port: 80
    targetPort: 8080
    protocol: TCP
```





4.4 Deploy to Kubernetes

Apply both manifests to your cluster:

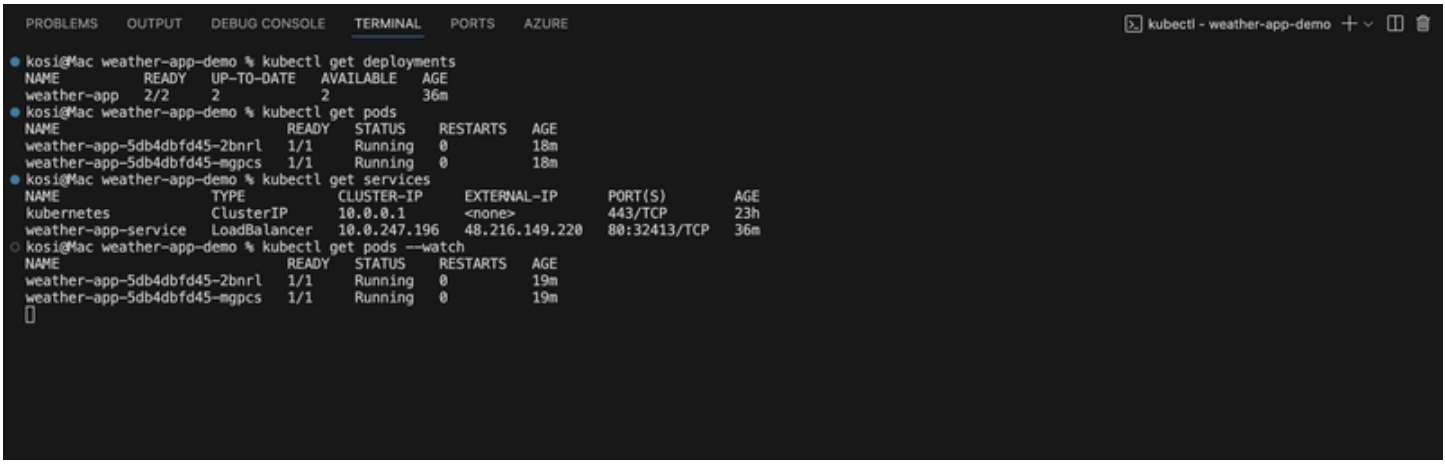
```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Check status:

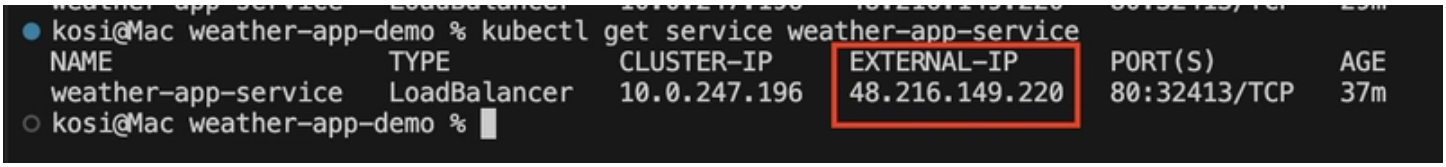
```
kubectl get deployments
kubectl get pods
kubectl get services
```

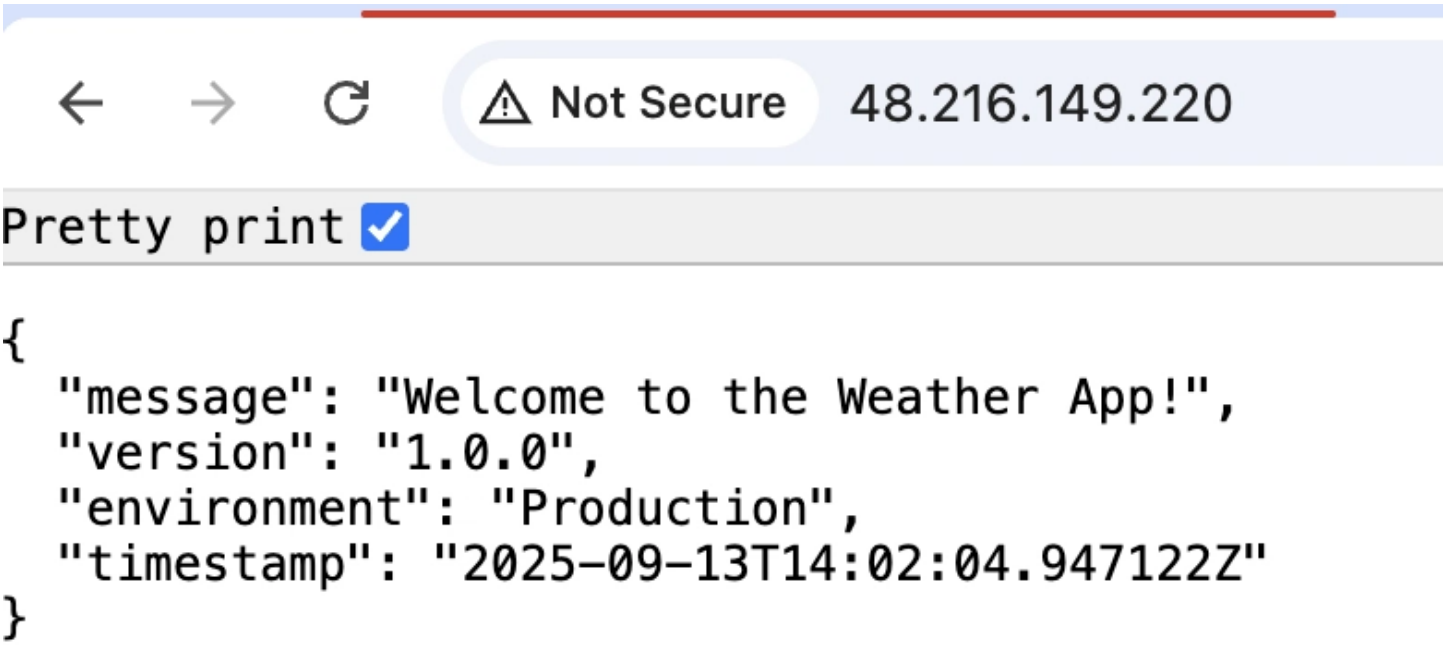
Watch pods come online:

```
kubectl get pods --watch
```



Once ready, your service will show an external IP, use that to access the app in your browser.





4.5 Troubleshooting Tips

If you see ImagePullBackOff, debug like this:

Describe pod to see error details

```
kubectl describe pod $(kubectl get pods -l app=weather-app -o
jsonpath='{.items[0].metadata.name}')
```

Verify the image exists in ACR

```
az acr repository show-tags --name studentdemo2024acr --repository weather-app --
output table
```

Restart deployment (forces a new image pull)

```
kubectl rollout restart deployment/weather-app
```

At this point, your app should be running on AKS and accessible from the public LoadBalancer IP.

Step 5: Set Up GitHub Actions CI/CD

With our app running in AKS, the final step is to automate deployment using GitHub Actions.
This way, every time you push changes to your repo, GitHub will:

Build the .NET app.

Build and push a Docker image to Azure Container Registry (ACR).

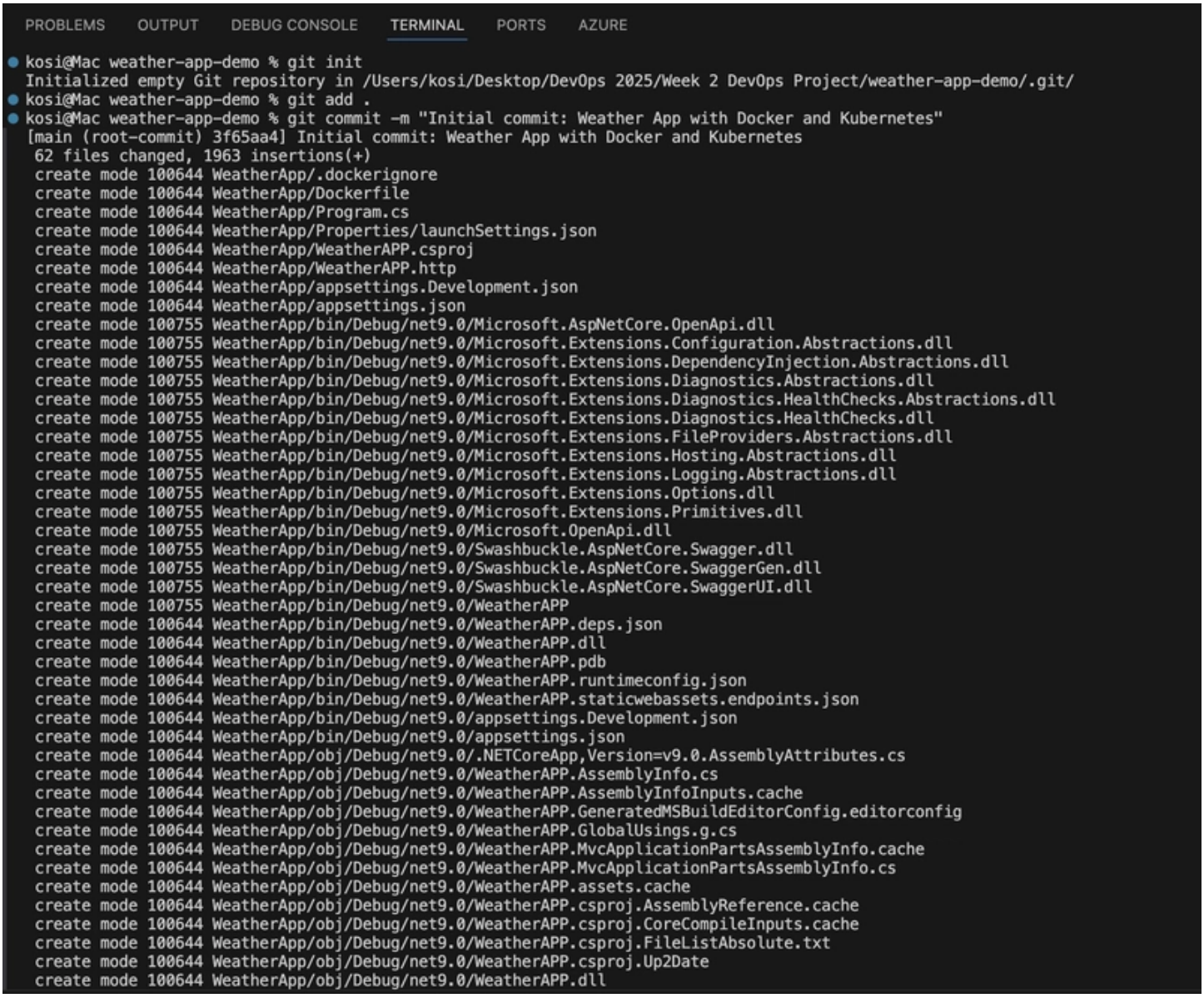
Update the Kubernetes deployment in AKS.

5.1 Initialize Git Repository

If you haven't already:

cd .. ### Back to weather-app-demo folder

```
git init
git add .
git commit -m "Initial commit: Weather App with Docker and Kubernetes"
```



5.2 Create Azure Service Principal

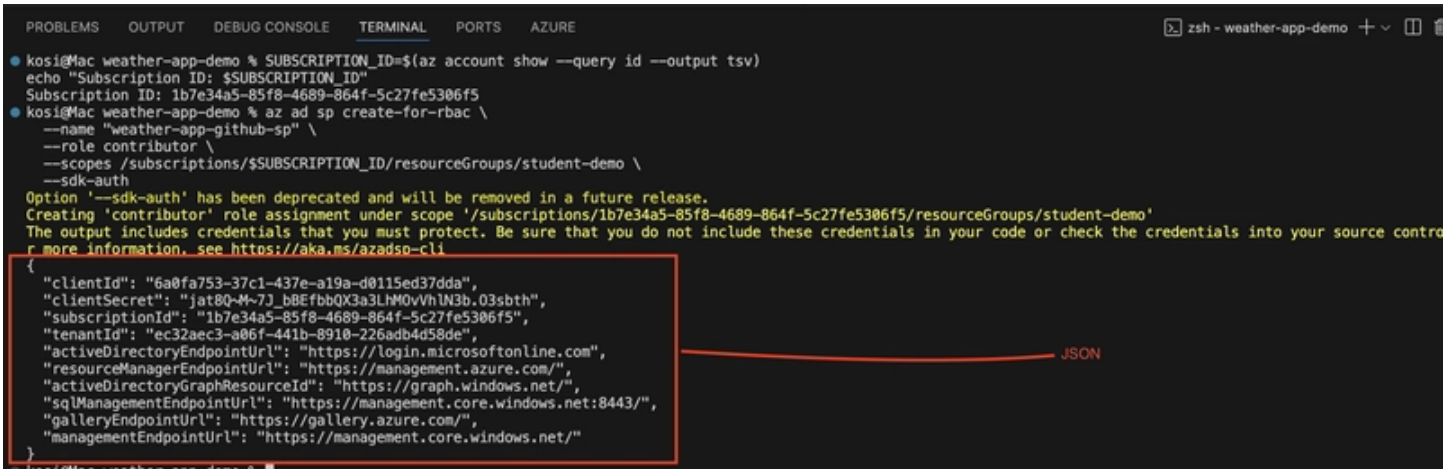
GitHub Actions needs credentials to interact with Azure. For that, we will create a Service Principal (SP).

Get your subscription ID:

```
SUBSCRIPTION_ID=$(az account show --query id --output tsv)
echo "Subscription ID: $SUBSCRIPTION_ID"
```

Create the service principal:

```
az ad sp create-for-rbac \
--name "weather-app-github-sp" \
--role contributor \
--scopes /subscriptions/$SUBSCRIPTION_ID/resourceGroups/student-demo \
--sdk-auth
```

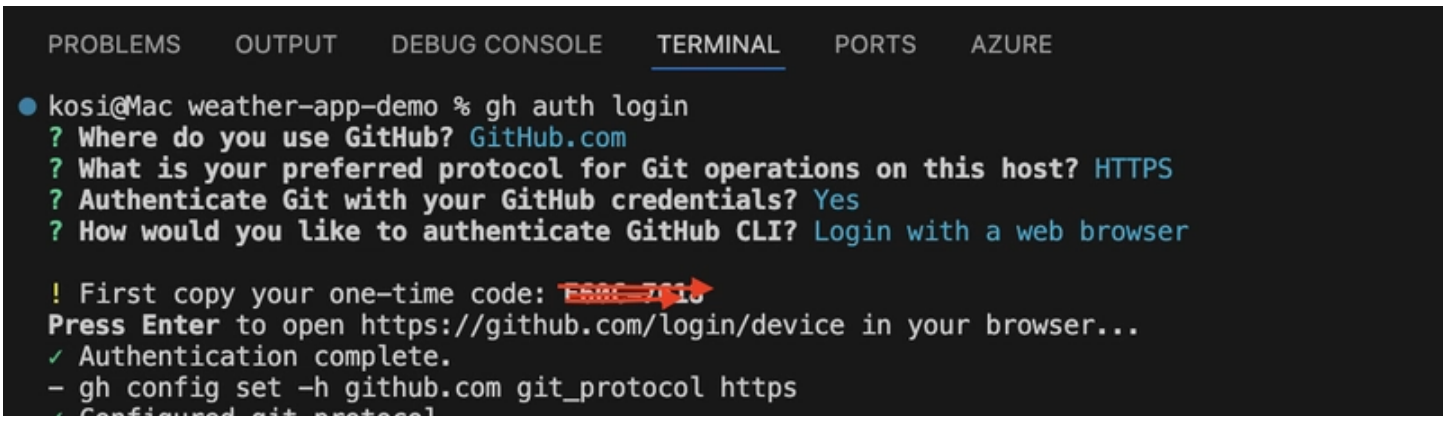


Important: Copy the full JSON output. You will paste it into GitHub Secrets in a moment.

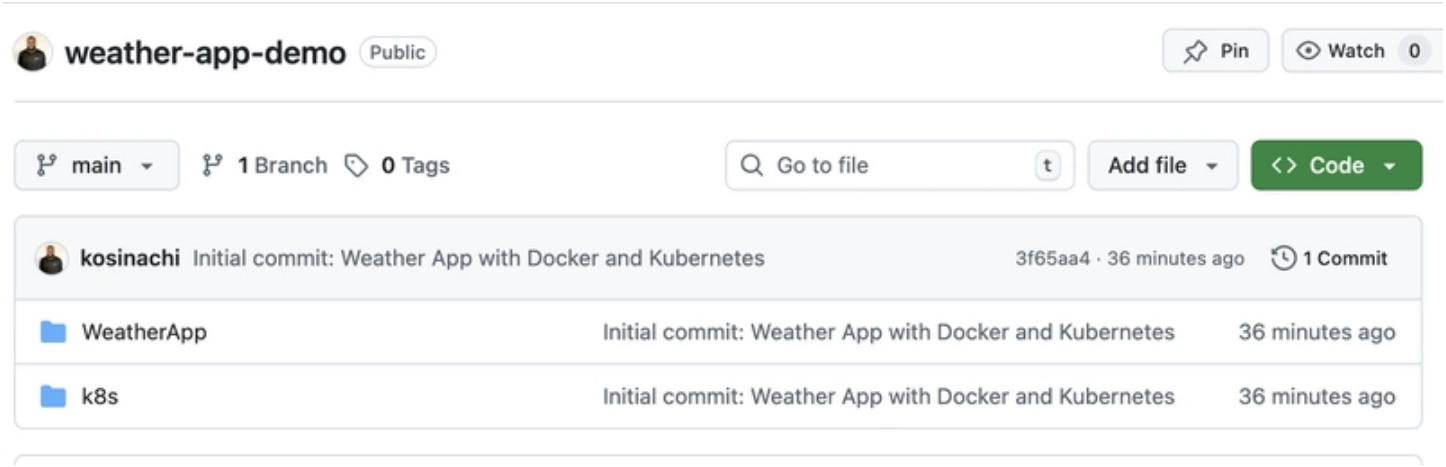
5.3 Create GitHub Repository

You can create a repo either via GitHub CLI:

```
gh auth login ### Follow the prompts
gh repo create weather-app-demo --public --source=. --push
```



```
✓ Configured git protocol
✓ Logged in as kosinachi

A new release of gh is available: 2.78.0 → 2.79.0
To upgrade, run: brew upgrade gh
https://github.com/cli/cli/releases/tag/v2.79.0

kosi@Mac weather-app-demo % gh repo create weather-app-demo --public --source=. --push
✓ Created repository kosinachi/weather-app-demo on github.com
  https://github.com/kosinachi/weather-app-demo
✓ Added remote https://github.com/kosinachi/weather-app-demo.git
Enumerating objects: 66, done.
Counting objects: 100% (66/66), done.
Delta compression using up to 10 threads
Compressing objects: 100% (59/59), done.
Writing objects: 100% (66/66), 1.12 MiB | 3.93 MiB/s, done.
Total 66 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (10/10), done.
To https://github.com/kosinachi/weather-app-demo.git
 * [new branch]      HEAD -> main
branch 'main' set up to track 'origin/main'.
✓ Pushed commits to https://github.com/kosinachi/weather-app-demo.git
kosi@Mac weather-app-demo %
```



Or manually on GitHub.com → then push your local repo.

5.4 Configure GitHub Secrets

In your repo, go to:

Settings → Secrets and variables → Actions → New repository secret

Add the following secrets:

```
Secret Name Value
AZURE_CREDENTIALS The full JSON output from Step 5.2
ACR_NAME studentdemo3121acr (your ACR name)
RESOURCE_GROUP student-demo
```

`CLUSTER_NAME student-aks-cluster`

OR configure in gh CLI

Set AZURE_CREDENTIALS (the JSON from step 5.2): `gh secret set AZURE_CREDENTIALS`

This will open an interactive prompt where you can paste your JSON credentials from step 5.2

Set the other secrets:

```
gh secret set ACR_NAME --body "studentdemo3121acr"
gh secret set RESOURCE_GROUP --body "student-demo"
gh secret set CLUSTER_NAME --body "student-aks-cluster"
```

5.5 Create GitHub Workflow

Now add the GitHub Actions workflow.

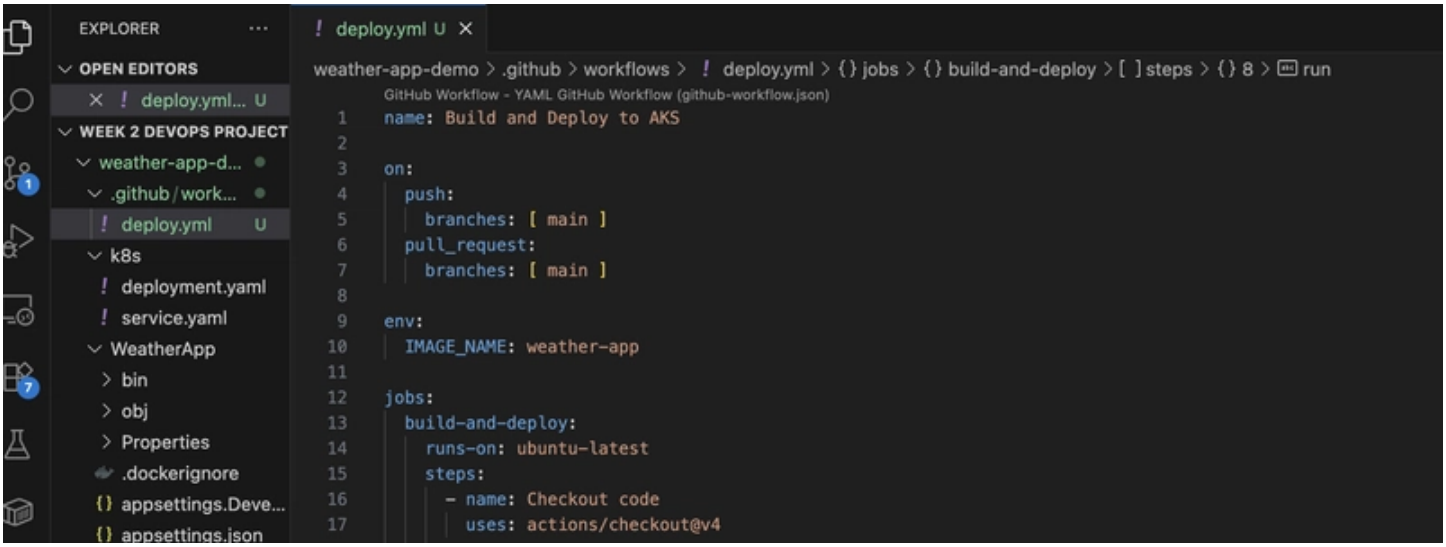
```
mkdir -p .github/workflows
touch .github/workflows/deploy.yml
```

Paste the code below into .github/workflows/deploy.yml:

```yaml
name: Build and Deploy to AKS
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  IMAGE_NAME: weather-app

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v3
        with:
          dotnet-version: '9.0.x'

      - name: Restore dependencies
        run: dotnet restore WeatherApp/WeatherAPP.csproj

      - name: Build application
        run: dotnet build WeatherApp/WeatherAPP.csproj --configuration Release --
no-restore

      - name: Run tests
        run: dotnet test WeatherApp/WeatherAPP.csproj --no-build --verbosity
normal || true

      - name: Azure Login
        uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}
```

```
      - name: Setup Azure CLI
        uses: azure/cli@v2
        with:
          inlineScript: echo "Azure CLI setup complete"

      - name: Build and push Docker image to ACR
        run: |
          IMAGE_TAG=${{ secrets.ACR_NAME }}.azurecr.io/${{ env.IMAGE_NAME }}:${{
github.sha }}
          az acr build \
            --registry ${{ secrets.ACR_NAME }} \
            --image ${{ env.IMAGE_NAME }}:${{ github.sha }} \
            --file WeatherApp/Dockerfile \
            .
          echo "IMAGE_TAG=$IMAGE_TAG" >> $GITHUB_ENV

      - name: Deploy to AKS
        if: github.ref == 'refs/heads/main'
        run: |
          echo "Checking if AKS cluster exists..."
          if ! az aks show --resource-group "${{ secrets.RESOURCE_GROUP }}" --
name "${{ secrets.CLUSTER_NAME }}" --output table; then
            echo "ERROR: AKS cluster '${{ secrets.CLUSTER_NAME }}' not found in
resource group '${{ secrets.RESOURCE_GROUP }}'"
            exit 1
          fi

          echo "Getting AKS credentials..."
          az aks get-credentials \
            --resource-group ${{ secrets.RESOURCE_GROUP }} \
            --name ${{ secrets.CLUSTER_NAME }} \
            --overwrite-existing

          echo "Testing kubectl connection..."
          kubectl cluster-info
```

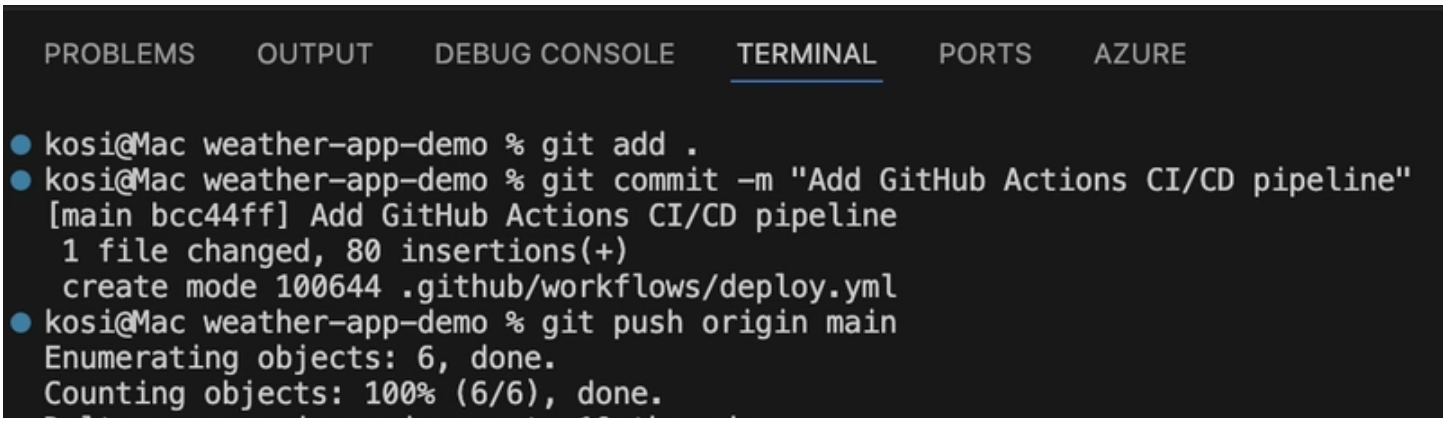
```
        echo "Updating deployment with image: ${{ env.IMAGE_TAG }}"
        kubectl set image deployment/weather-app \
          weather-app=${{ env.IMAGE_TAG }}

        echo "Waiting for rollout to complete..."
        kubectl rollout status deployment/weather-app --timeout=600s

        echo "Deployment status:"
        kubectl get pods -l app=weather-app
        kubectl get service weather-app-service
```



5.6 Deploy Your Application

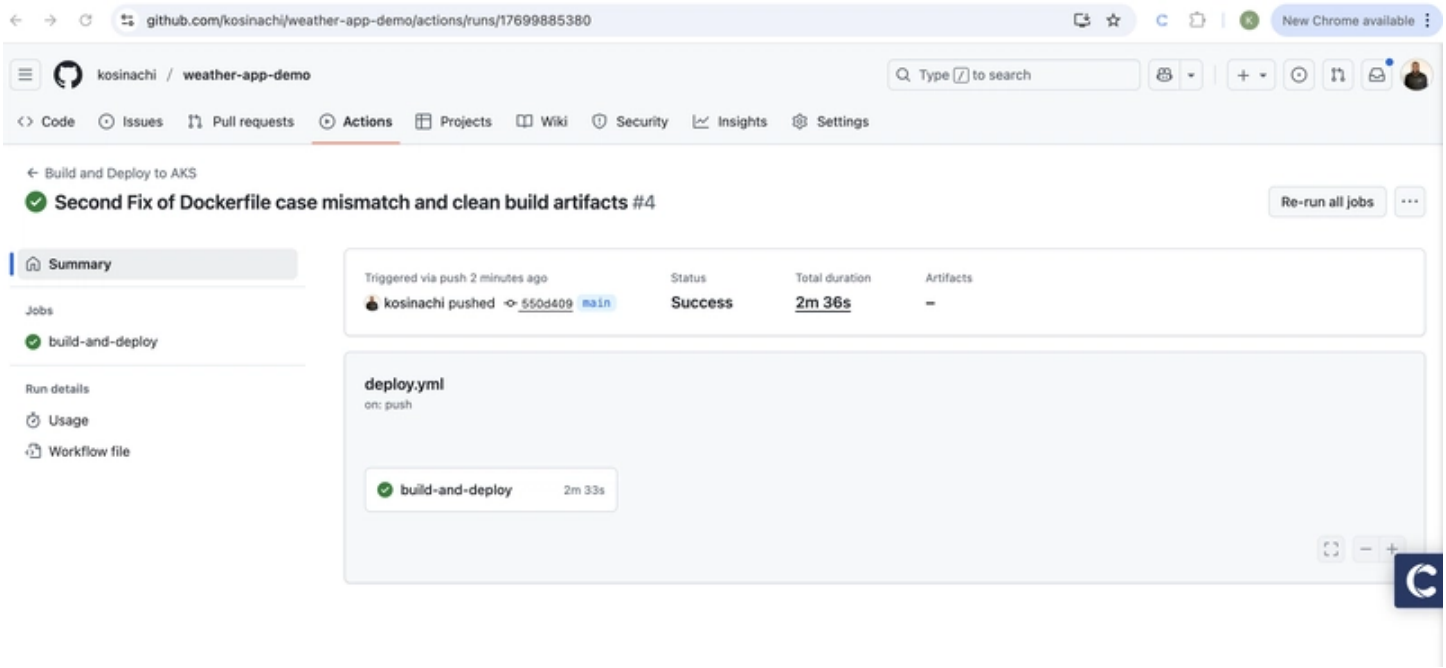
Commit and push:

```
git add .
git commit -m "Add GitHub Actions CI/CD pipeline"
git push origin main
```



```
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 1.34 KiB | 1.34 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kosinachi/weather-app-demo.git
   3f65aa4..bcc44ff  main -> main
kosi@Mac weather-app-demo %
```

Now open your GitHub repo → Actions tab.
You will see the workflow running, building your Docker image, pushing to ACR, and updating your AKS cluster.



And just like that, we have built a full CI/CD pipeline for your .NET 8 app on Azure Kubernetes Service.

Step 6: Access Your Deployed Application

Your app is now deployed on Azure Kubernetes Service. Let's get the public URL so you can test it live.

6.1 Get the External IP Address

Check the status of your Kubernetes service:

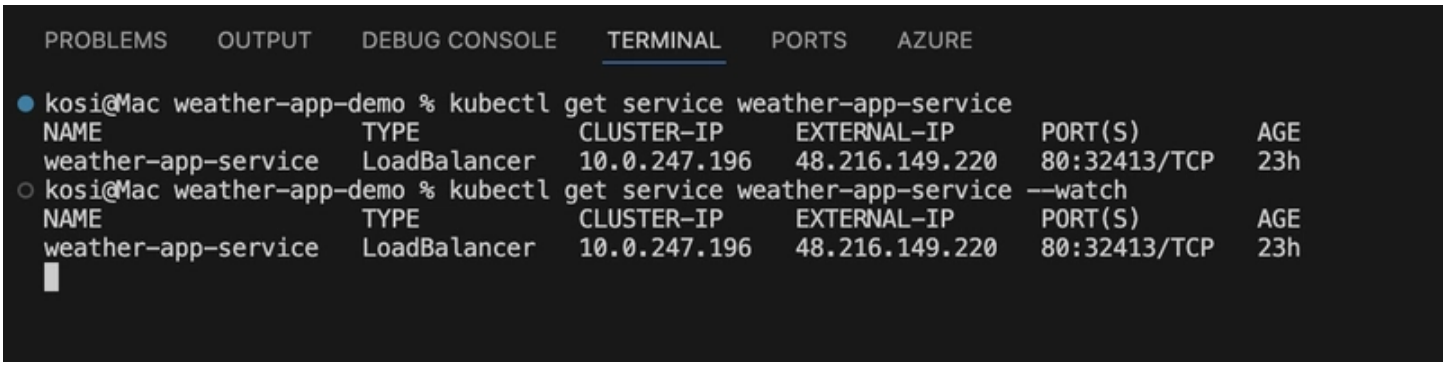
```
kubectl get service weather-app-service
```

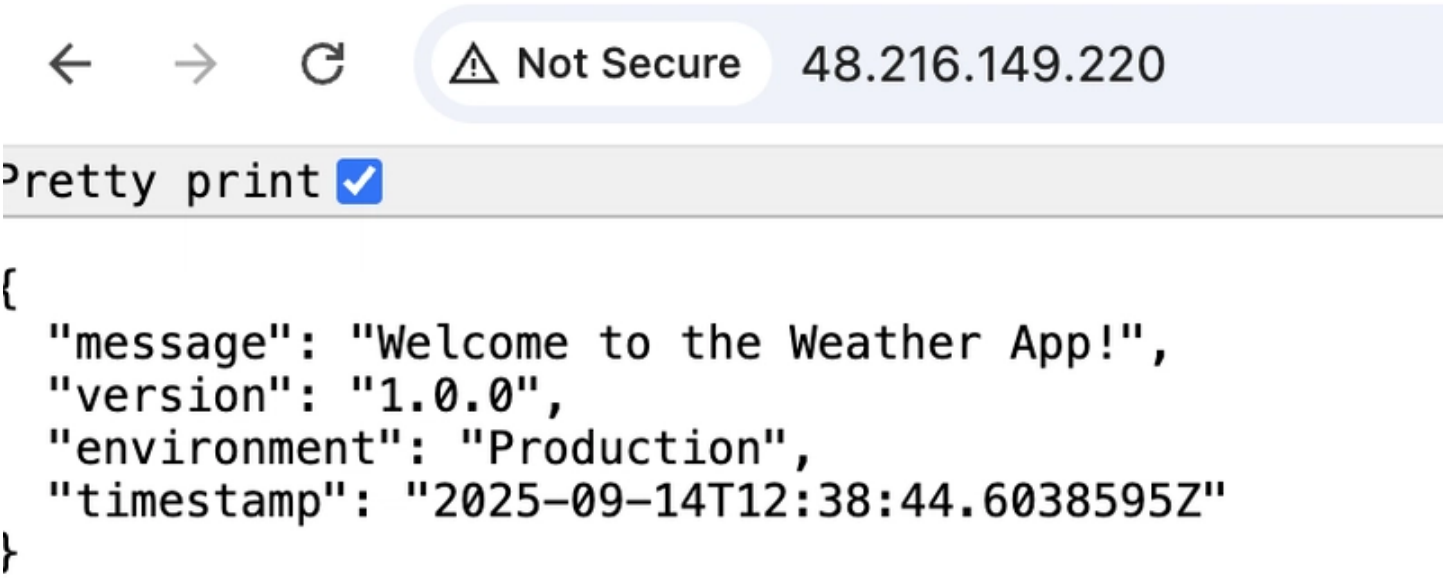
You will see output like this:

NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
weather-app-service LoadBalancer 10.0.123.45 20.85.102.11 80:31000/TCP 2m

👉 The EXTERNAL-IP column is what you will use.
It may take 2–5 minutes to show up. Watch until it's assigned:

```
kubectl get service weather-app-service --watch
```





6.2 Test Your Live Application

Once you have the external IP, test the endpoints:

For the Welcome message:

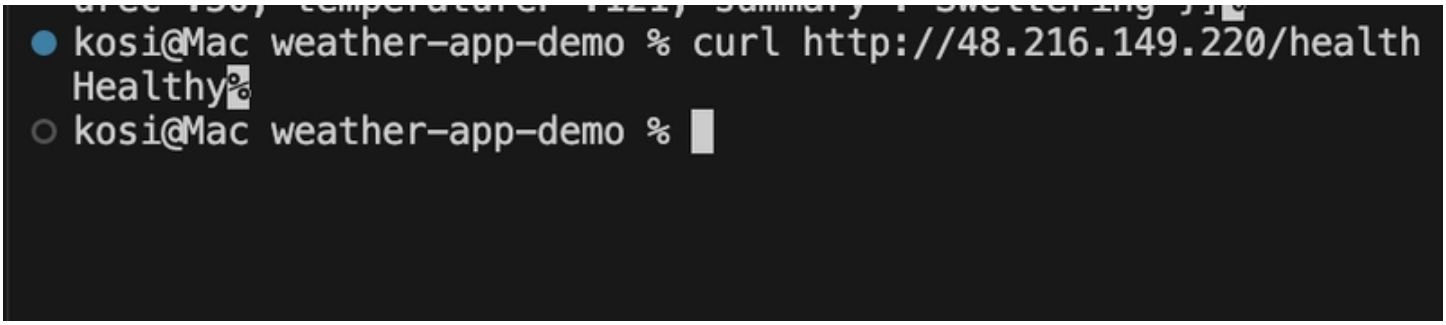
```
curl http://YOUR-EXTERNAL-IP/
```

Weather forecast:

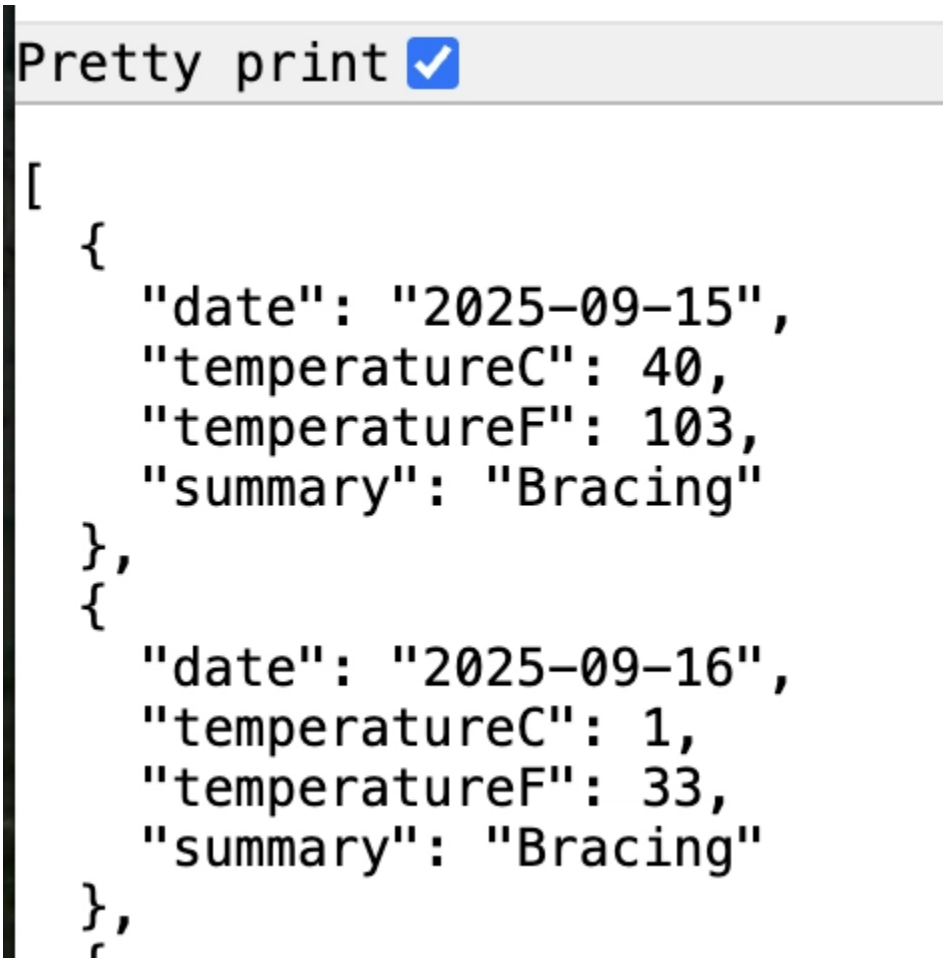
```
curl http://YOUR-EXTERNAL-IP/weather
```

Health check:

```
curl http://YOUR-EXTERNAL-IP/health
```



Or just open it in your browser:



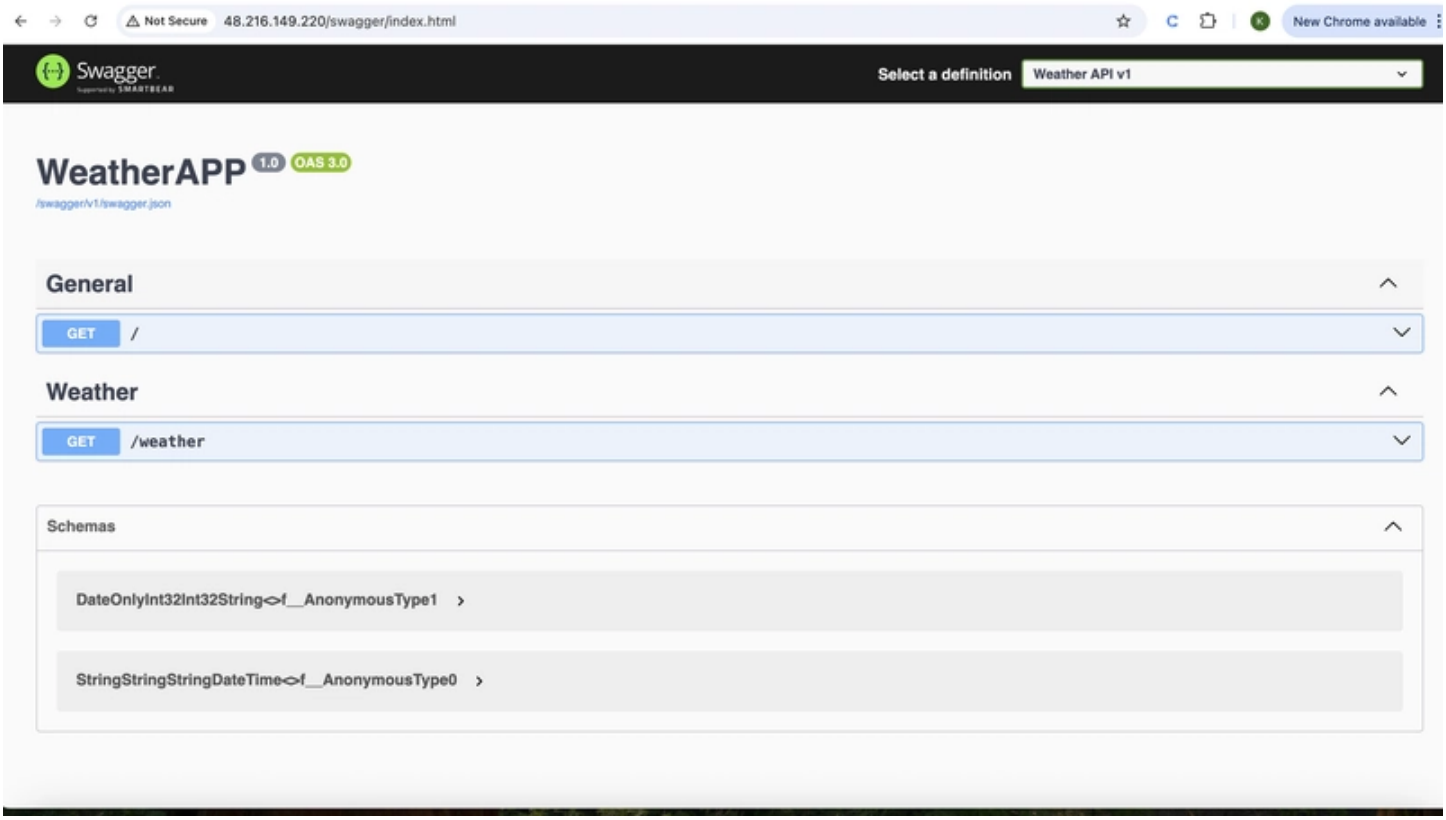
```
  {
    "date": "2025-09-17",
    "temperatureC": 52,
    "temperatureF": 125,
    "summary": "Scorching"
  },
  {
    "date": "2025-09-18",
    "temperatureC": 40,
    "temperatureF": 103,
    "summary": "Scorching"
  },
  {
    "date": "2025-09-19",
    "temperatureC": 7,
    "temperatureF": 44,
    "summary": "Chilly"
  }
]
```


Healthy

Swagger UI (API docs):

`open http://YOUR-EXTERNAL-IP/swagger ### macOS`
`start http://YOUR-EXTERNAL-IP/swagger ### Windows`

In macOS



✅ Congratulations! Your .NET 8 Weather API is now fully containerized, deployed on Kubernetes, and publicly accessible through Azure AKS.

Step 7: Test Continuous Deployment

Now let's see GitHub Actions automatically deploy changes to AKS with no manual intervention required.

7.1 Make a Code Change

Edit `WeatherApp/Program.cs` to update the welcome message, for example:

```
app.MapGet("/", () => new
{
    Message = "Welcome to the Updated Weather App! 🌤️",
    Version = "1.1.0",
    Environment = app.Environment.EnvironmentName,
```

```
    Timestamp = DateTime.UtcNow,
    DeployedBy = "GitHub Actions"
});
```

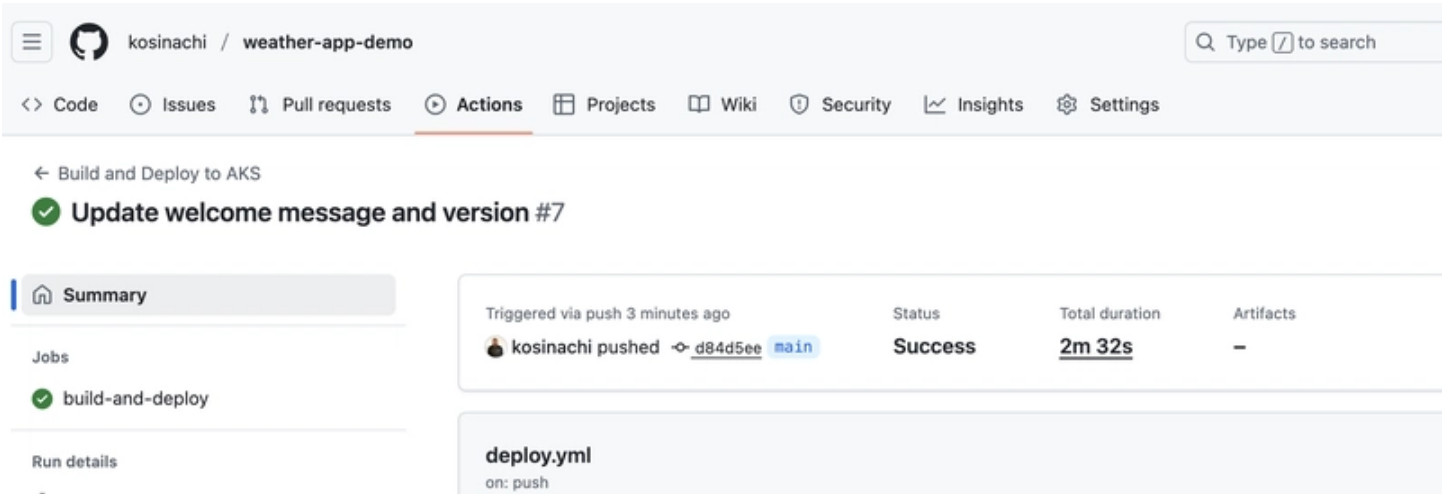
This small change is perfect to test the pipeline.

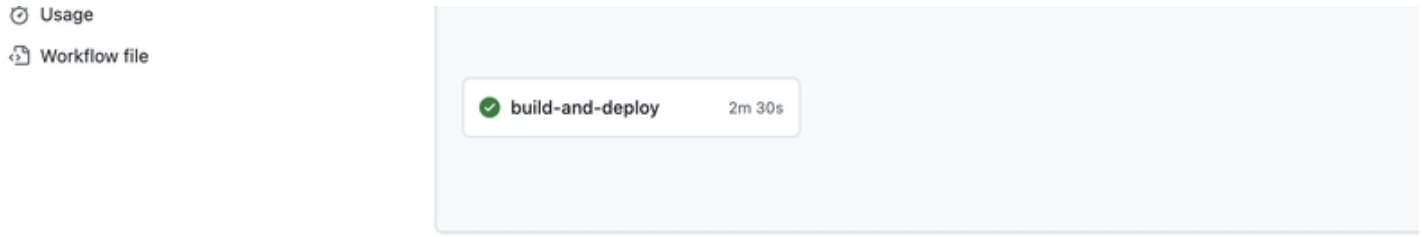
7.2 Push the Changes

```
git add .
git commit -m "Update welcome message and version"
git push origin main
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE

kosi@Mac WeatherApp % cd ..
kosi@Mac weather-app-demo % git add WeatherApp/Program.cs
kosi@Mac weather-app-demo % git commit -m "Update welcome message and version"
[main d84d5ee] Update welcome message and version
 1 file changed, 49 insertions(+), 9 deletions(-)
kosi@Mac weather-app-demo % git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 10 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.23 KiB | 1.23 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/kosinachi/weather-app-demo.git
   76ce73b..d84d5ee  main -> main
kosi@Mac weather-app-demo %
```

Pushing to the main branch triggers your GitHub Actions workflow automatically.





7.3 Verify Auto-Deployment

Open your GitHub repository → Actions tab.

Watch the workflow run:

Builds the .NET app

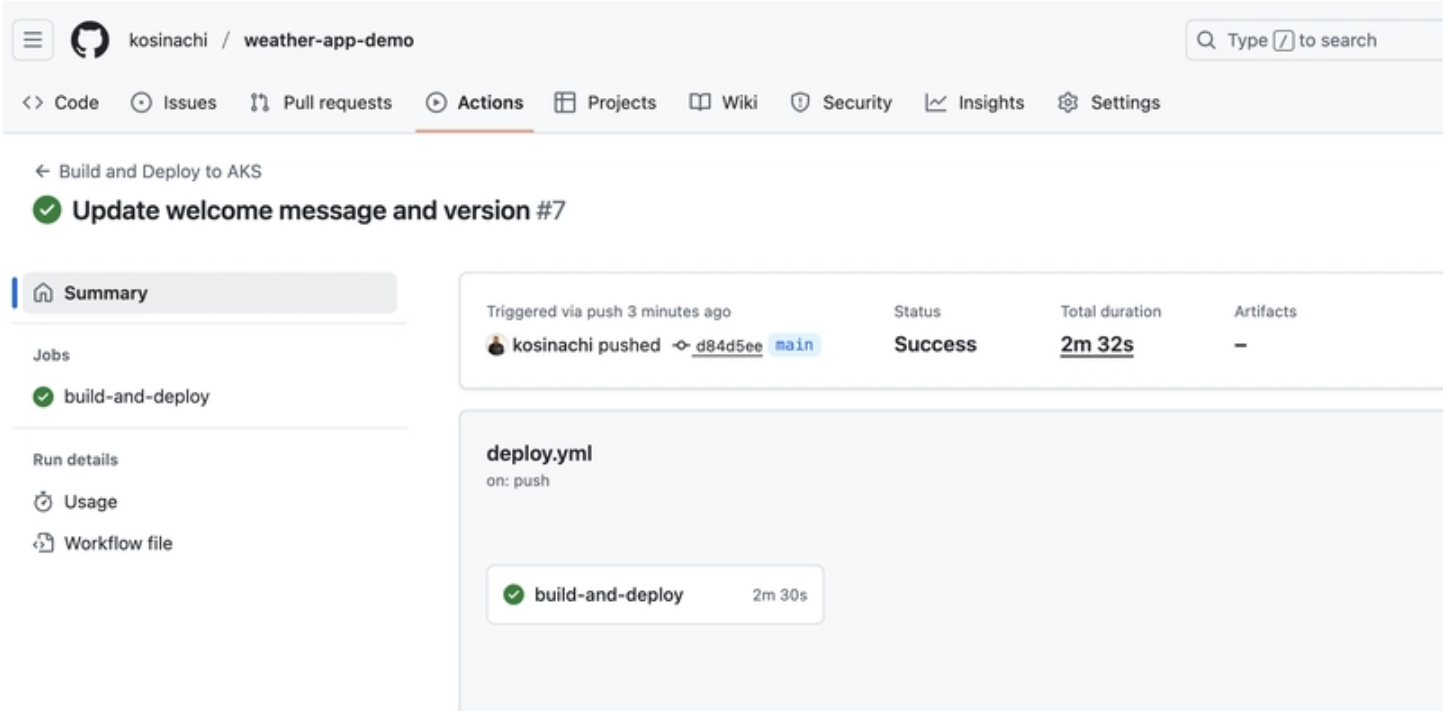
Builds & pushes the Docker image to ACR

Updates the Kubernetes deployment in AKS

Once completed, test your updated app:

```
curl http://YOUR-EXTERNAL-IP/
```

You should see the new welcome message and updated version, confirming that CI/CD is working flawlessly as confirmed below.



✅ Congratulations! we have now have a fully functional CI/CD pipeline:

Push code → automatically builds, pushes, and deploys

Changes reflected live in your AKS cluster

All without manually logging into Azure

Troubleshooting Guide

Even with the best setup, things can go wrong. Here are some common issues you might face and how to fix them.

Pods stuck in `ImagePullBackOff` or `ErrImagePull`

Symptoms:
Your pods won't start and show ImagePullBackOff.

Fixes:

```
# Check pod details
kubectl describe pod $(kubectl get pods -l app=weather-app -o
jsonpath='{.items[0].metadata.name}')

# Verify AKS can access your ACR
az aks check-acr \
  --resource-group student-demo \
  --name student-aks-cluster \
  --acr studentdemo2024acr.azurecr.io

# Re-attach ACR if needed
az aks update \
  --resource-group student-demo \
  --name student-aks-cluster \
  --attach-acr studentdemo2024acr

# Force pod recreation
kubectl delete pods -l app=weather-app
```

2. kubectl: command not found

Symptoms:
kubectl isn't recognized.

Fixes:

```
# Check if installed
kubectl version --client
```

If not installed, follow the official guide:
👉 Install kubectl

3. Docker daemon not running

Symptoms:
Docker commands fail with "Cannot connect to the Docker daemon."

Fixes:

Start Docker Desktop

Wait until the green status indicator shows

Restart your terminal

4. Access denied to Azure resources

Symptoms:
Azure CLI commands fail with "unauthorized" errors.

Fixes:

```
az logout
az login
az account show
```

Verify you are using the correct subscription.

5. GitHub Actions pipeline failing

Check these first:

All GitHub secrets are correctly set (AZURE_CREDENTIALS, ACR_NAME, etc.)

The Azure service principal has Contributor access to your resource group

Inspect the GitHub Actions logs for detailed error messages

6. Application not accessible via External IP

Fixes:

```
# Check pods
kubectl get pods

# Check service
kubectl get services

# Inspect pod logs
kubectl logs deployment/weather-app

# Describe the service
kubectl describe service weather-app-service

**Debugging Commands**

These are useful one-liners when things don’t go as planned:

# Cluster info
kubectl cluster-info

# All resources in the namespace
kubectl get all

# Pod logs
kubectl logs -l app=weather-app

# Deployment details
kubectl describe deployment weather-app

# Node status
kubectl get nodes -o wide

# Monitor pods in real-time
kubectl get pods --watch

# ACR repositories
az acr repository list --name studentdemo2024acr --output table
```

```
# ACR image tags
az acr repository show-tags \
  --name studentdemo2024acr \
  --repository weather-app \
  --output table
```

✅ With these troubleshooting steps, you should be able to resolve most common deployment issues quickly.

Clean Up Resources

Once you are done experimenting, don't forget to clean up your Azure resources otherwise, you may keep paying for unused services.

You have two options:

Option 1: Delete Individual Resources

If you want to keep the resource group but remove specific resources:

Delete Kubernetes resources (Deployment + Service)

```
kubectl delete -f k8s/
```

Delete AKS cluster

```
az aks delete \
--resource-group student-demo \
--name student-aks-cluster \
--yes
```

Delete Azure Container Registry (ACR)

```
az acr delete \
--name studentdemo3121acr \
--yes
```

Option 2: Delete Everything (Recommended)

If this was just a demo project, the easiest and safest way is to delete the entire resource group:

Delete the entire resource group (removes AKS, ACR, etc.)

```
az group delete \
--name student-demo \
--yes \
--no-wait
```

⚠ Note: Your GitHub repository and workflow remain intact — only Azure resources will be deleted.

What we have Accomplished

By following along, we have gone from zero to cloud-native hero. Here's what we have built:

A Production-Ready .NET 8 Web API

RESTful endpoints with proper HTTP responses

Health checks for monitoring

Swagger documentation for easy API testing

Mastered Docker Containerization

Multi-stage Docker builds for optimization

Container security best practices

Local testing and validation

Deployed to Azure Kubernetes Service (AKS)

Infrastructure as Code with Azure CLI

Kubernetes manifests with proper resource requests/limits

LoadBalancer service for external access

Implemented CI/CD with GitHub Actions

Automated build and test pipeline

Container registry integration (ACR)

Zero-downtime deployments with rolling updates

Applied Cloud-Native Best Practices

Health, readiness, and startup probes for reliability

Resource limits for efficiency

Environment-specific configurations

📚 Next Steps

Now that you have got the foundation, here's how you can level up:

Beginner Level

Add a Database: Integrate Azure SQL or PostgreSQL, use EF Core for persistence

Enhance Monitoring: Add Application Insights, set up log aggregation, track custom metrics

Implement Security: Add authentication with Azure AD, enable API versioning, and secure with HTTPS

Intermediate Level

Advanced Kubernetes Features: ConfigMaps & Secrets, Horizontal Pod Autoscaling, Ingress controllers with custom domains

Infrastructure as Code: Learn Azure Bicep or Terraform, adopt GitOps with ArgoCD, manage multiple environments

Observability Stack: Prometheus & Grafana for metrics, Jaeger for distributed tracing, ELK stack for centralized logging

You now have the skills to build, ship, and run modern .NET apps on the cloud like a pro. From here, the sky's the limit, scale it, secure it, and make it production-grade!

Azure Kubernetes Service (AKS) Deployment - Complete Guide Sheet

Whether you are deploying your first .NET app to Kubernetes or need a quick refresher, this cheat sheet gives you everything you need from Docker to AKS to GitHub Actions.

Core Concepts Explained

⚡ Kubernetes

Container Orchestrator: Manages your containers like a traffic controller

Scaling: Add/remove pods based on load

Self-Healing: Restarts crashed pods automatically

Load Balancing: Routes traffic evenly across replicas

🐳 Docker

Containerization: Package app + dependencies

Consistency: "Works on my machine" → "Works everywhere"

Isolation: Each container runs independently

📦 Azure Container Registry (ACR)

Private Docker Hub: Secure container storage

Version Control: Track different image versions

Integration: Works seamlessly with AKS

🛠️ Prerequisites
Tool Purpose Why You Need It
VS Code Code Editor Write and edit code
.NET 8 SDK Build Tool Compile your C# app
Docker Desktop Containers Build & test locally
Azure CLI Cloud Management Create/manage Azure resources
kubectl Kubernetes Client Deploy & manage apps on AKS
GitHub CLI Repo Management Create and push repos
Git Version Control Track code changes

Accounts Needed

GitHub → Free for repos & CI/CD

Azure → $200 free credit to host apps

Azure Infrastructure

📂 Resource Group

```
az group create --name student-demo --location eastus`
```

⛶

Resource group: Think of it as a project folder for all Azure resources

Container Registry (ACR)

```
az acr create --resource-group student-demo --name studentdemo2024acr --sku Basic
```

⛶

Container registry is like an app store, where things can be stored, share, and download ready-to-run software packages called containers.
Think of it like your private Docker Hub

AKS Cluster

```
az aks create --resource-group student-demo --name student-aks-cluster --node-count 1
```

This is like a managed Kubernetes cluster with auto-scaling & healing

.NET App Breakdown

🔑 Program.cs

Health Checks → `app.MapHealthChecks("/health");`

Swagger → API testing UI

Endpoints → `/`, `/weather`, `/health`

Kestrel Config

```
builder.WebHost.ConfigureKestrel(options => {
    options.ListenAnyIP(8080);
});
```

Runs on port 8080 inside container

Docker Setup

📜 Dockerfile (multi-stage build)

Base → runtime image

Build → SDK to restore & publish

Final → small, production-ready image

.dockerignore

Ignore `bin/`, `obj/`, `.git/` → smaller, faster, more secure builds

Kubernetes Manifests

📦 deployment.yaml

replicas: 2 → high availability

resources.requests/limits → efficient pod usage

health probes → auto-healing + traffic control

service.yaml

```
type: LoadBalancer
ports:
  - port: 80
    targetPort: 8080
```

This exposes app with an external IP

GitHub Actions Workflow

🔄 CI/CD Pipeline Highlights

Checkout Code → `actions/checkout@v4`

Setup .NET → build/test app

Azure Login → via service principal

Build & Push Image → to ACR

Deploy to AKS → update deployment, restart rollout

Secrets Required

`AZURE_CREDENTIALS` → Service principal JSON

`ACR_NAME` → Your ACR name

`RESOURCE_GROUP` → Azure resource group

`CLUSTER_NAME` → AKS cluster name

Monitoring & Observability

Health Checks → `HTTP 200` = healthy

Probes:

Liveness → restart if dead

Readiness → remove from traffic if unready

Startup → delay checks until app boots

Troubleshooting Essentials

Check Deployment → `kubectl rollout status deployment/weather-app`

Pod Logs → `kubectl logs <pod-name>`

Service Info → `kubectl describe service weather-app-service`

Image Issues → `az acr repository show-tags --name studentdemo2024acr --repository weather-app`

Quick Reference Commands

Docker

```
docker build -t app:tag .
docker run -p 8080:8080 app:tag
docker logs container-name
```

Kubernetes

```
kubectl apply -f k8s/
kubectl get pods
kubectl logs -f <pod-name>
kubectl scale deployment weather-app --replicas=3
```

Azure CLI

```
az aks get-credentials --resource-group student-demo --name student-aks-cluster
az acr login --name studentdemo2024acr
az group delete --name student-demo --yes
```

🔥 With this guide sheet which i call cheat sheet, you have all the key commands, configs, and concepts at your fingertips for deploying .NET apps on AKS with GitHub Actions.